

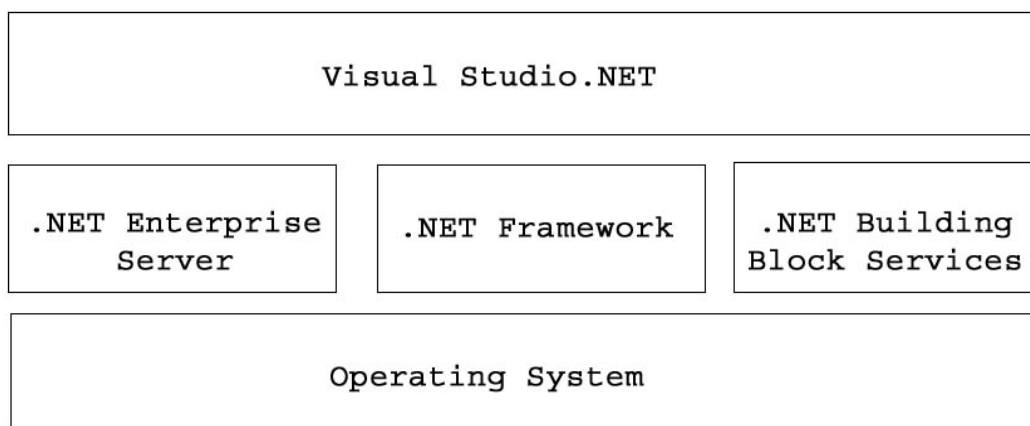
บทที่ 2

หลักการและทฤษฎี

2.1 Microsoft .NET Technology

2.1.1 .NET PLATFORM

องค์ประกอบของ แพลตฟอร์ม .NET แบ่งออกได้เป็น 3 ชั้น



รูปที่ 2-1 องค์ประกอบ .NET Platform

2.1.1.1 ชั้นระบบปฏิบัติการ (Operating System)

2.1.1.2 ชั้นถัดมาแบ่งออกเป็น 3 ส่วน ได้แก่

- .NET Enterprise Server
 - SQL Server 2000, BizTalk Server, Exchange 2000
- .NET Framework
 - เป็น Framework ที่สำคัญในการพัฒนา .NET
- .NET Building Block Services
 - Microsoft passport, Notification and Messaging, Personalization , XML Store, Calendar, Directory and Search

2.1.1.3 ชั้น Visual Studio .NET เครื่องมือในการพัฒนา .NET

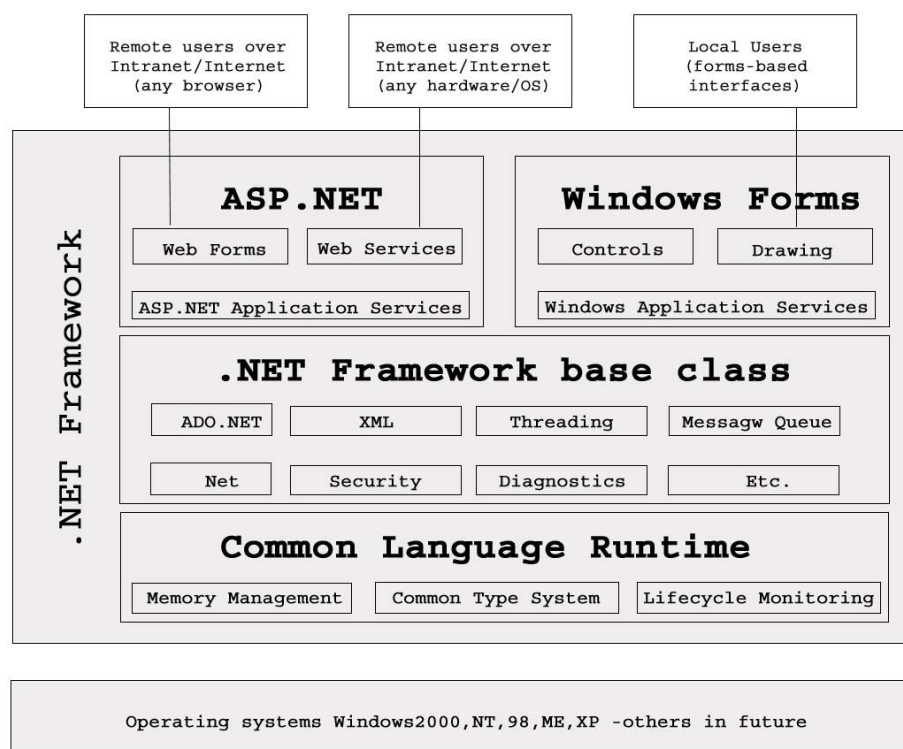
2.1.2 .NET Framework

.NET Framework มีองค์ประกอบ ดังนี้

1. Common Language Runtime (CLR) มีหน้าที่ในการจัดเตรียมบริการต่างๆที่เกี่ยวข้องเพื่อให้โปรแกรมที่ขอใช้บริการสามารถทำงานได้

2. .NET Framework Class Library (Unified programming classes) คือไฟล์ไลบรารีที่ทำหน้าที่จัดเก็บและรวบรวมข้อมูลที่เป็นต้องเรียกใช้งาน ในการพัฒนาโปรแกรม

3. ASP.NET เป็น Web application model ซึ่งประกอบไปด้วย กลุ่มของ control และ Infrastructure ที่ใช้ในการสร้าง ASP Web applications , ASP.NET ประกอบไปด้วยส่วนของ control ซึ่งรวมถึงส่วนของ HTML ด้วย ส่วนของ control รันบนเซิร์ฟเวอร์ ส่วน HTML รันบน browser บนเซิร์ฟเวอร์นั้นส่วน control จะถูกเขียนในรูปแบบของ object-oriented programming , ASP.NET ยังให้บริการส่วน infrastructure ด้วยเช่น session state management และ process recycling ยิ่งไปกว่านั้น ASP.NET ก็ใช้คอนเซปต์นี้ในการพัฒนา จาก software เป็น service โดยผ่าน กลไกสำคัญคือ XML เราสามารถเขียน business logic และใช้ ASP.NET infrastructure เพื่อให้บริการ ผ่าน SOAP



รูปที่ 2-2 องค์ประกอบ .NET Framework

2.1.2.1 Common Language Runtime (CLR)

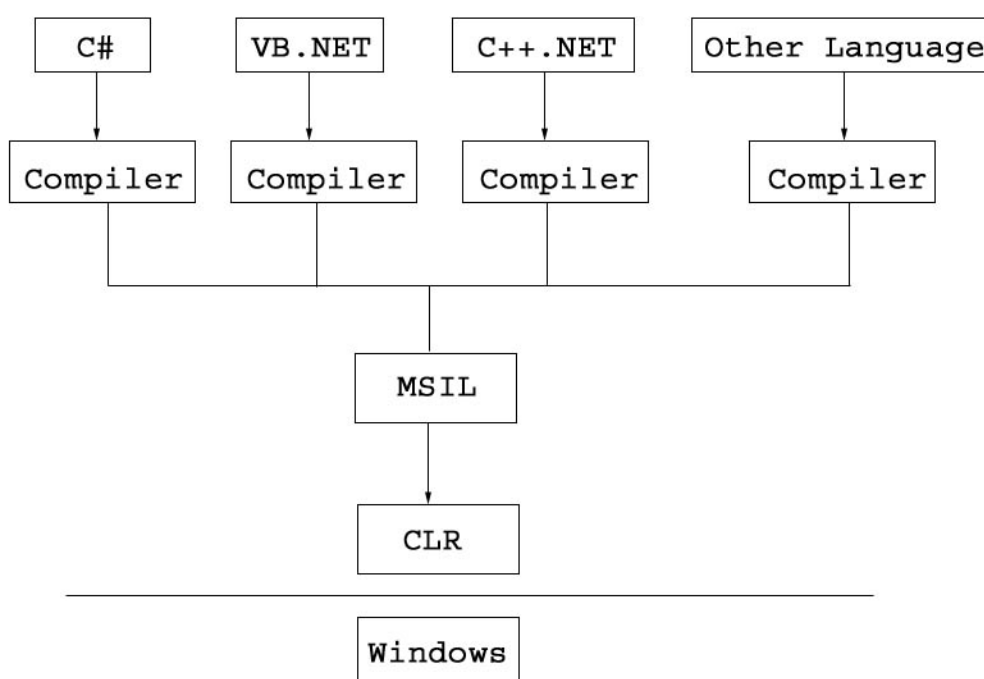
CLR จะทำหน้าที่เป็นตัวกลางในการจัดเตรียมบริการและทรัพยากรสำหรับรองรับการประมวลผล และการทำงานของโปรแกรมประยุกต์ที่ทำงานบนเทคโนโลยีของ .NET เช่น การจัดการหน่วยความจำ, ความปลอดภัยในการเข้ารหัสโปรแกรม ด้วยความสามารถของ CLR ทำให้เราสามารถพัฒนาโปรแกรมโดยไม่ต้องขึ้นกับระบบปฏิบัติการ

การทำงานของโปรแกรมนั้น เริ่มจากคอมไพเลอร์ของแต่ละภาษาจะคอมไพล์โค้ดให้เป็นแบบ Microsoft's Intermediate Language (MSIL) หรือเรียกสั้นๆว่า IL Code ซึ่งจะมีลักษณะคล้ายภาษา Assembly จาก IL Code ก็แปลงเป็นโปรแกรมที่รันโดย CLR อีกทีหนึ่ง

นักพัฒนาสามารถที่จะเลือกได้ว่า เราจะ build แอปพลิเคชันไปอยู่ในรูปของ .exe หรือ MSIL ซึ่งจะกลายเป็น Just-In-Time(JIT) คือเมื่อเราได้แอปพลิเคชันในรูปแบบของ MSIL แล้ว เมื่อรันโปรแกรมใช้งานจริง มันจะถูก คอมไพเลอร์ JIT ทำการคอมไพล์โค้ด MSIL ในส่วนที่ต้องการใช้ไปเป็น Native Code อีกทีซึ่งนำไปให้เครื่องรันต่อ หากมีการใช้โค้ดในส่วนเดิมอีกก็ไม่ต้องมีการคอมไพล์ซ้ำ

การแปลงโค้ด MSIL ไปเป็น Native Code ประโยชน์ที่เราจะได้คือ

1. เราสามารถแปลงไปเป็น Native Code ที่เหมาะสมกับระบบปฏิบัติการนั้นรันอยู่
2. เราสามารถได้ Native Code ที่ใช้ความสามารถของ CPU ได้อย่างเต็มที่ เช่นใน CPU P4 เราสามารถใช้คำสั่งในส่วนของ SSE2 ได้



รูปที่ 2-3 รูปแบบการคอมไพล์โค้ดไปเป็น IL Code

Managed Code	เป็นโค้ดที่คอมไพล์ให้ทำงานใน .NET Framework
Native Code	เป็นโค้ดที่คอมไพล์เป็นภาษาเครื่องโดยตรง ไม่ต้องผ่านตัว CLR ในการรันโปรแกรม
Unmanaged Code	เป็นโค้ดที่ไม่ได้คอมไพล์เพื่อใหทำงานใน .NET Framework เช่น COM Object ใน version ก่อนหน้านี้ ซึ่ง .NET ก็ยังสามารถรันโค้ดพวกนี้ได้ แต่ไม่สามารถใช้ความสามารถของ .NET ได้อย่างเต็มที่

2.1.2.1.1 Memory Management

สำหรับ .NET แล้วจะมี Garbage Collector (GC) ซึ่งทำหน้าที่คอยรวบรวมหาหน่วยความจำที่ถูกทิ้งไว้เป็นขยะ แต่ไม่มีใครอ้างอิงเรียกใช้แล้ว โดยการทำงานจะแตกต่างจากใน VB Runtime รุ่นเก่าที่ให้ Object รับผิดชอบในการจัดการ Garbage Collector เป็นผู้ดูแลจัดการทั้งหมด โดย GC จะเรียก method ชื่อ Object.Finalize โดยอัตโนมัติเมื่อ object เลิกใช้งาน

การจัดการหน่วยความจำที่ดีกว่าของ .NET เกิดขึ้นเมื่อ object ถูกสร้างขึ้นมา CLR จะจัดการกันพื้นที่ส่วนหนึ่งไว้สำหรับโปรแกรมที่เรียกว่า Heap โดยเริ่มต้นจากพื้นที่หน่วยความจำที่ว่าง จากนั้นก็อ้างอิงชี้ตำแหน่งไปยังส่วนบนสุดของหน่วยความจำ เมื่อหน่วยความจำถูกใช้แล้วยกเลิกไปเรื่อยๆ จะเหลือพื้นที่ว่างเป็นช่วงๆ ซึ่ง GC จะเข้ามาทำการ Compact หน่วยความจำเสียใหม่ ลองนึกถึงหลักการทำงานแบบเดียวกันกับตอนที่ทำ Defragment บนฮาร์ดดิสก์

2.1.2.1.2 Common Type System

ใน .NET จะมี common Type System ที่จะให้ทุกภาษามี Types ที่เหมือนกัน เป็นมาตรฐาน โดยที่ทุก Types ที่สนับสนุนโดย Common Types System นั้น จะสืบทอดคุณสมบัติมาจาก System.object ดังนั้นคุณจะได้พบว่า Object ที่หลายส่วนมากแล้วจะสนับสนุน Method เหล่านี้คือ

Equal(Object) = Boolean

GetHashCode() = Int32

GetType () = Type

Tostring () =String

ข้อดีของ Common Language Runtime

- ไม่มีปัญหาเรื่องการรัน บน MS Windows Platform ต่างๆ เนื่องจาก CLR จะทำการตรวจสอบระบบปฏิบัติการให้โดยอัตโนมัติ และทำการจำลองสภาพให้เหมาะสมกับการทำงานของโปรแกรมโดยอัตโนมัติเช่นเดียวกัน

- ไม่ต้องสนใจเรื่อง Registry การเข้าถึง component ต่างๆที่เกี่ยวข้องจะอยู่ในความดูแลของ CLR ทั้งหมด เราจำเป็นต้องรู้แค่จะใช้ Namespace ตัวใดในการอ้างอิงถึงเท่านั้น

- รองรับการพัฒนาจากหลายภาษา ในปัจจุบันเราสามารถใช้อย่างน้อยสามภาษามาทำงานร่วมกันได้ด้วยเทคโนโลยีของ COM แต่อย่างไรก็ตาม เนื่องจากโปรแกรมยังผูกติดกับภาษาอยู่ ความเร็วของโปรแกรมที่ได้ก็จะแตกต่างกัน ขึ้นอยู่กับความสามารถของ Compiler ภาษานั้นๆ Microsoft Common Language Specification จะทำให้ compiler ทุกตัวที่ใช้ .NET ทำการแปลภาษาไปยังเป้าหมายเดียวกัน คือการทำงานกับ CLR ได้อย่างไร้ปัญหา ส่งผลให้ โปรแกรมที่ได้จากและภาษามีผลลัพธ์ที่เท่าเทียมกันเมื่อทำงานอยู่ภายใต้การดูแลของ CLR

- นำ Source code มาใช้งานใหม่ได้ COM ยังถูกจำกัดการ reuse ในวงแคบเนื่องจากการผูกติดกับระบบปฏิบัติการ .NET แยกออกได้ 3 รูปแบบได้แก่

1. เราสามารถเขียน class ด้วยภาษาหนึ่ง สามารถเรียกใช้ผ่านอีกภาษาหนึ่งได้
2. คลาสสามารถ Inherit ข้ามภาษาได้
3. Web Service เป็นอีกรูปแบบหนึ่งโดยที่เสนอบริการให้แก่ทุกภาษาที่อ้างอิง CLR ได้
4. ชนิดของข้อมูลมีประสิทธิภาพ จากข้อผิดพลาดเกี่ยวกับตัวแปรเช่น ประกาศตัวแปรขนาด 10 ไบต์ แต่ทำงานจริงกลับมีขนาด 20 ไบต์ ถ้าไม่มีการเขียนโปรแกรมดักจับ โปรแกรมจะทำงานผิดพลาด ใน .NET สิ่งเหล่านี้จะไม่เกิดขึ้น CLR จำตรวจสอบการทำงานของ code อย่างใกล้ชิดก่อนที่จะประมวลผล ขั้นสุดท้าย CLR จะไม่อนุญาตให้โปรแกรมใช้ตัวแปรที่มีการผิดพลาดเหล่านั้น
5. การ Debug ที่มีประสิทธิภาพ CLR จะทำการแยกส่วนของโปรแกรมเป็นส่วนๆทำให้การตรวจสอบข้อผิดพลาดทำได้ง่ายสะดวกและรวดเร็ว การแก้ไขสามารถแก้ไขเฉพาะส่วนที่ผิดพลาดได้โดยไม่รบกวนส่วนอื่นๆของโปรแกรม และเนื่องจาก CLR จัดการกับทุกภาษาด้วยกัน ข้อผิดพลาดต่างๆที่แสดงออกมาในแต่ละภาษาจะอยู่ในรูปแบบเดียวกัน
6. ระบบจัดสรรทรัพยากรที่ดีขึ้น การดำเนินการที่เกี่ยวข้องกับทรัพยากรต่างๆ ระหว่างโปรแกรม ไม่ว่าจะเป็นระบบการจัดการไฟล์ระบบการเชื่อมต่อ หรือพื้นที่ว่างของหน้าจอ เป็นต้น จะถูกดูแลจัดการอย่างใกล้ชิด โดย CLR ไม่สามารถมีโปรแกรมตัวใดตัวหนึ่งครอบครองทรัพยากรไว้แต่เพียงผู้เดียว
7. การแจกจ่ายโปรแกรมง่ายขึ้น การติดตั้งโปรแกรมแบบเดิม โปรแกรมหนึ่งๆมีไฟล์เป็นจำนวนมาก ,ต้องมีการลงทะเบียนใน registry ,การสร้าง short cut , การจัดเตรียมระบบที่เกี่ยวข้อง ใน .NET การติดตั้งโปรแกรมเป็นไปโดยง่าย เนื่องจาก CLR จะทำการแยกโปรแกรมออกเป็นส่วนๆ การติดตั้งโปรแกรมจึงเป็นการ copy ไฟล์ที่เกี่ยวข้องมาไว้บน harddisk เท่านั้น ในทางกลับกัน การยกเลิกการติดตั้งก็เพียงลบไฟล์เหล่านั้นออก
8. ระบบความปลอดภัยที่ดีขึ้น ระบบรักษาความปลอดภัยใหม่นี้จะทำการตรวจสอบพฤติกรรมการทำงานของโปรแกรมอย่างใกล้ชิด อย่างในระบบเดิมนั้น การทำงานของ ภาษาสคริปต์ต่างๆหรือ โปรแกรมที่มาจาก Internet อาจเป็นโปรแกรมที่สร้างช่องโหว่ให้กับระบบได้

2.1.2.2 .NET Framework Class Library

ไฟล์ไลบรารีที่ทำหน้าที่จัดเก็บและรวบรวมข้อมูลที่จำเป็นต้องเรียกใช้งาน ในการพัฒนาโปรแกรม ซึ่งไฟล์เหล่านี้ เป็นไลบรารีร่วมกันในทุกภาษา ส่งผลให้เราสามารถ inheritance ,error handling, debugging ข้ามภาษาได้ และการที่ .NET Framework เสนอบริการสำเร็จรูปผ่าน Class Library เหล่านี้ทำให้ รูปแบบการเขียนโปรแกรมเปลี่ยนไป โดยมีความเป็นระบบ และเป็นลักษณะของ OOP เพิ่มขึ้น เช่นใน VB.NET ต้องมีการอ้างอิงถึง class library ที่เกี่ยวข้องและต้องการใช้งาน

2.1.2.3 Presentation Layer

การแสดงผลสามารถทำได้ 2 รูปแบบหลัก คือในรูปแบบ Windows form และ Web form ซึ่งก่อนนี้ทั้งคำสั่ง และกระบวนการเขียนโปรแกรม ของทั้ง 2 แบบ แต่ใน .Net แล้วได้รวมทั้งสองอย่างมาทำให้เหมือนกันที่สุด คือเราเขียนโปรแกรมบน Windows อย่างไร คุณก็เขียน Web Application ด้วยวิธีเดียวกัน โดยเรียกใช้ Class ด้วยคำสั่งที่เหมือนกัน บน Windows จะอยู่ใน class ที่ชื่อ system.windows แต่ Web จะอยู่ใน class ชื่อ system.web.ui ซึ่งวิธีใช้เหมือนกันมาก ที่พิเศษคือ มี Object สำหรับ Web ให้คุณ

เขียนโปรแกรม แบบ visual basic ธรรมดา แต่เมื่อรันบน browser แล้วจะสร้าง tag html และ javascript ให้อัตโนมัติ ทำให้ browser ทั่วๆ ไปก็สามารถรันได้ ไม่จำเป็นต้องสนับสนุน .net โดยเฉพาะ ตัวอย่างเช่น เราสร้างเว็บเพจที่มี ปฏิทิน ด้วย(นึกถึง calendar ใน VB) ในส่วนของ รูปแบบ web จะแสดง ในรูปของ html code ส่วนปฏิทินจะอยู่ในรูปของ java script ซึ่งทางโปรแกรมจะทำการแปลงให้เอง

2.1.3 .NET Compact Framework

.NET Compact Framework คือ platform ของ Microsoft .NET ที่ใช้ในการพัฒนาอุปกรณ์เล็กๆ ความจริงแล้ว .NET Compact Framework ก็คือ .NET Framework ที่ตัดเอาส่วนที่ไม่สำคัญหรือไม่ได้ใช้ออกไป เพื่อทำให้ขนาด เล็กลงจนขนาดที่สามารถ install ลงบนอุปกรณ์เล็กๆ ที่ไม่ใช่คอมพิวเตอร์ได้ เช่น โทรศัพท์มือถือ เป็นต้น

คุณสมบัติหลักของ .NET Compact Framework

- .NET Compact Framework ถูกออกแบบให้เป็น platform ในการพัฒนา สำหรับการเขียน และเรียกใช้ XML Web Services
- เนื่องจาก .NET Compact Framework ใช้ programming model เดียวกันกับ desktop .NET Framework จึงง่ายต่อ developer ในการเขียน application ใหม่บน .NET Compact Framework
- ใช้ Visual Studio .NET ตัวเดียวกับที่ใช้ใน desktop .NET Framework ในการเขียน program device application
- ใช้ระบบ evidence-base security ที่อนุญาตให้เฉพาะ application ที่ผ่านการรับรองจากผู้ผลิต device นั้น จึงจะสามารถ access secure resource ได้ ถ้าเป็น application ที่ยังไม่ผ่านการรับรองความปลอดภัย จะ access ได้เพียง basic system resource เท่านั้น
- ออกแบบสำหรับอุปกรณ์ที่มี resource จำกัด เช่น โทรศัพท์มือถือ โดยที่ตัว framework เองใช้ memory อย่างมีประสิทธิภาพ และมีระบบคืน resource ที่ไม่ได้ใช้แล้วกลับมาจาก application อย่างมีประสิทธิภาพ
- ใช้ Just-in-time compiler ซึ่งมีประสิทธิภาพดีกว่า ระบบ device programming อื่นๆ ที่ใช้ code interpreter
- เพิ่มประสิทธิภาพและลดต้นทุนในการพัฒนา เนื่องจากใช้ programming model และ toolset ของ Visual Studio .NET อันเดียวกันกับของ desktop .NET framework ทำให้นักพัฒนาสามารถใช้ความรู้ความชำนาญที่มีอยู่แล้วมาพัฒนา application บน อุปกรณ์ได้

2.1.4 .NET Framework on Linux

ไมโครซอฟท์ ทำการส่ง spec มาตรฐานของ .net framework ให้แก่ ECMA เพื่อสร้างเป็นมาตรฐานสากล ส่งผลให้ .NET Framework เป็น Language Independent และ Platform Independent ตอนนี้บริษัท XIMIAN ทำการสร้าง framework บน Linux แล้ว ประกอบไปด้วย CLR , BASE CLASS , C# Compiler

2.1.4.1 .NET Framework Runtime (CLR)

Ximian ตั้งชื่อ project ในการทำ .NET Framework บน Linux ว่า Mono โดยจะมีหลักอยู่ 3 โปรแกรม ดังนี้

1. mint โปรแกรมนี้คือโปรแกรม .NET Framework Runtime บน Linux โดยสามารถเอาโปรแกรมที่เขียนโดยใช้ .NET Framework บน Windows มาใช้งานได้เลย
2. pedump เป็นโปรแกรมที่ดูโครงสร้างของแฟ้ม .exe ที่เป็น Managed code
3. monodis เป็นโปรแกรมที่แสดงให้เห็นถึงภาษา MSIL ของ .exe นั้น

2.1.4.2 NET Framework based class Library

การ Implement Library ของ .NET นั้น มีความซับซ้อนยิ่งกว่าส่วนอื่นๆ หากสนใจสถานะว่า Library ตัวใดทำไปแล้วมากน้อยเพียงใด สามารถหาข้อมูลได้จาก <http://www.go-mono.com/class-status/index.html>

2.1.4.3 C# Compiler

Ximian ทำคอมไพเลอร์ภาษา C# ที่ชื่อว่า mcs (Mono CSharp compiler)

2.1.5 มาตรฐาน CLI และ C# ของ ECMA

เดือนสิงหาคม ปี2000 ไมโครซอฟท์ ฮีเลดต์แพกการ์ด และ อินเทล ได้ร่วมเป็นผู้สนับสนุนร่วมเสนอการกำหนดรายละเอียดสำหรับ CLI และ ภาษา C# ให้กับองค์กรการจัดมาตรฐานระหว่างประเทศ ECMA ผลคือ ECMA ได้ตั้งมาตรฐานกลุ่มการทำงานขึ้นมา 2 กลุ่ม สำหรับ CLI และ C# คือ TC39-TG3 และ TC39-TG2 ตามลำดับซึ่งเป็นคณะกรรมการเทคนิคซึ่งรับผิดชอบเกี่ยวกับภาษาในการเขียนโปรแกรมและการพัฒนาแอปพลิเคชัน

ระหว่างปีถัดมาผู้ให้การสนับสนุนร่วมร่วมกับสมาชิก ECMA อื่นๆ และ ผู้รับเชิญ (รวมถึง IBM Fujitsu Software, Plum Hall, Monash University และ ISE) ได้ร่วมทำการกลั่นกรองรายละเอียด เป็นมาตรฐานในธันวาคมปี 2001 สมัชชาสหประชาชาติ ECMA อนุมัติ มาตรฐานฉบับที่1 ของ C# และ CLI เป็น ECMA-334 และ ECMA-335 ตามลำดับ รายงานเกี่ยวกับ CLI ได้ถูกอนุมัติเป็นมาตรฐาน ECMA TR84 ด้วย

อ้างอิงจาก

<http://msdn.microsoft.com/net/ecma>

<http://www.ecma-international.org/>

2.2 XML

2.2.1 XML คืออะไร

เนื่องจากทุกวันนี้โลกของเรามีการนำเทคโนโลยีสารสนเทศเข้ามาใช้ในชีวิตประจำวันมากขึ้น และได้เข้ามาเป็นส่วนหนึ่งของการทำงาน ที่เห็นได้ชัดคือ เทคโนโลยีบนอินเทอร์เน็ต ไม่ว่าจะเป็นการเข้าไปเยี่ยมชมเว็บไซต์ต่าง ๆ หรือการส่งจดหมายอิเล็กทรอนิกส์ที่เรียกว่า อีเมล ซึ่งกำลังเป็นมาตรฐานของการติดต่อสื่อสารสำหรับอนาคต ทำให้ต้องมีการคิดเพื่อพัฒนาให้มีความก้าวหน้ามากขึ้น การเขียนเว็บไซต์ในปัจจุบันนี้ โดยปกติแล้วสิ่งที่จะใช้สร้างเว็บได้คือ ภาษา HTML ที่เรารู้จักกันดีเท่านั้น แต่ในวันนี้โลกได้มีการพัฒนาเทคโนโลยีใหม่ที่ใช้สำหรับการเขียนเว็บ นั่นคือ XML ซึ่งเป็นสิ่งที่หลายผลิตภัณฑ์ให้การสนับสนุน

Extensive Markup Language (XML) เป็นภาษาที่ให้ความชัดเจนในการให้รายละเอียดเกี่ยวกับข้อมูล และการเปลี่ยนแปลงข้อมูลโดยแอปพลิเคชันบนเว็บและใช้ฟอร์มที่ยืดหยุ่นได้ตามมาตรฐาน HTML หรือ Hyper Text Markup Language ได้เปิดโลกแห่งการแสดงผลข้อมูลต่างๆ มานานแสนนาน ส่วน XML จะทำให้การทำงานกับข้อมูลโดยตรงที่เสริมกับการทำงานของ HTML

2.2.1.1 ความเป็นมาของ XML (Extensive Markup Language)

โพรโทคอลอินเทอร์เน็ต (อินเทอร์เน็ต Protocol-IP), HypertText Markup Language และ HyperText Transport Protocol (HTTP) ได้เป็นการปฏิวัติและสร้างมิติใหม่ในการกระจายข้อมูลและสารสนเทศ การนำเสนอ ตลอดจนการค้นคืน โดยให้ผู้ใช้สามารถใช้สารสนเทศที่ต้องการได้ง่ายด้วยบราวเซอร์ และมี search engine หรือเครื่องมือในการช่วยค้นหา นอกจากนี้ยังมีการประยุกต์ไปใช้กับเครือข่ายในสำนักงานหรืออินเทอร์เน็ต และใช้สำหรับการบริการข้อมูลสำหรับลูกค้าและคู่ค้าที่สามารถตอบสนองทางด้านสารสนเทศที่ต้องการ ได้อย่างมีประสิทธิภาพมากขึ้น

สำหรับ Extensive Markup Language จะให้ประโยชน์อย่างเต็มที่เมื่อทำงานร่วมกับ HTML ด้วยเหตุที่ว่า XML ได้มีความพร้อมในแง่ของรายละเอียด และการนำข้อมูลตลอดจนโครงสร้างข้อมูลมาแสดงได้ในรูปแบบ Text ผ่านทาง HTTP ที่เปิดให้ข้อมูลขึ้นใหม่และมีความสามารถในการจัดข้อมูลได้อีกด้วยในการเขียนเว็บเพจเมื่อใช้ HTML ผู้พัฒนาสามารถกำหนดได้ว่าส่วนไหนจะเป็นตัวหนา ตัวเอียง หรือตัวอักษรเป็นแบบไหน ส่วน XML นั้นจะเป็นการเตรียมส่วนของข้อมูลที่จะนำไปใส่ในช่องที่กำหนดตามการเขียนของ HTML ยกตัวอย่างเช่น ข้อมูลด้านราคา หรือราคาที่ตั้งสำหรับการจัดรายการส่งเสริมการขาย อัตราภาษี ค่าขนส่ง เป็นต้น

XML ถือได้ว่าเป็นส่วนหนึ่งของ Standard Generalized Language Markup Language (SGML) ที่เป็นข้อกำหนดในการสร้างหรือจัดทำเอกสารในรูปแบบอิเล็กทรอนิกส์ที่กำหนดโดย W3C หรือ World Wide Web Consortium สามารถดูรายละเอียดเพิ่มเติมได้จาก <http://www.w3.org/TR/REC-xml> ที่มีโครงสร้างและรูปแบบที่เปิดให้แอปพลิเคชันต่างๆ สามารถเรียกไปใช้งานได้ เช่น บนเว็บไซต์ต่างๆ เป็นต้น และทางไมโครซอฟท์ได้มีการทำงานร่วมกับ W3C เพื่อพัฒนามาตรฐานข้อมูลบนเว็บที่ทำให้ HTML สามารถแสดงผลข้อมูลที่ XML ได้เตรียมไว้ และทางไมโครซอฟท์เองได้มีการเปิดตัว บราวเซอร์ตั้งแต่ IE 4.0

เป็นต้นไป ที่สามารถเรียกดูและประมวลผลข้อมูลได้ และเป็นข้อกำหนดให้ บราวเซอร์เวอร์ชันใหม่ของค่ายไมโครซอฟท์สนับสนุน XML

สิ่งที่ถือได้ว่าเป็นลักษณะเด่นของ XML นั้นจะเป็นความสะดวกในการจัดการด้านระบบการติดต่อกับผู้ใช้จากโครงสร้างของข้อมูล เราสามารถนำข้อมูลจากหลายแหล่งมาแสดงผลและประมวลผลร่วมกันได้ ไม่ว่าจะเป็นข้อมูลลูกค้า รายการสั่งซื้อ ผลการวิจัย รายการรับชำระเงินข้อมูลเวชระเบียน รายการสินค้าหรือข้อมูลสารสนเทศอื่นๆ ก็สามารถแปลงให้เป็น XML ได้ และในส่วนของข้อมูลสามารถปรับให้เป็น HTML ได้

สำหรับประโยชน์ในการใช้งานนั้น เราจะสามารถนำมาใช้สำหรับการเข้าถึงระบบข้อมูลขนาดใหญ่ใช้กับระบบเครือข่ายในองค์กร หรืออินเทอร์เน็ตเพื่อดูข้อมูลหรือเรียกใช้ข้อมูลที่ทำให้การแสดงผลทางหน้าจอที่รวดเร็วและง่ายในการจัดการ

XML (Extensive Markup Language) คืออะไร

Extensive Markup Language เป็นฟอร์แมตที่อธิบายถึงรายละเอียดของโครงสร้างและแบบของข้อมูลเป็นภาษาหรือชุดคำสั่งเกี่ยวกับข้อมูลบนเว็บ ที่ให้การพัฒนาและมีศักยภาพในส่วนของโครงสร้างข้อมูลจากหลากหลายแอปพลิเคชันมาเสนอบนเครื่องเดสก์ทอป ด้วย XML จะทำให้การจัดการข้อมูลหรือเรียกใช้ข้อมูลจากแอปพลิเคชันต่างๆ จะเข้าสู่มาตรฐานเดียวกัน

XML จะเป็นส่วนหนึ่งของ HTML ซึ่ง XML จะให้รายละเอียดเกี่ยวกับข้อมูล เช่น ชื่อเมือง อุณหภูมิ ความกดอากาศ ส่วน HTML เป็นการกำหนด tag ต่างๆ ที่จะทำให้ข้อมูลแสดงออกมาในรูปแบบไหน ซึ่งข้อมูลจะสามารถแสดงออกมาได้หลายรูปแบบ ไม่ว่าจะเป็นตารางหรือข้อความธรรมดา ขึ้นอยู่กับการกำหนดของ HTML และในปัจจุบันนี้ ด้วย XML จะมีการให้รายละเอียดของเนื้อหาเอกสารที่เรียกว่า Document Type Definition (DTD) ที่ให้รายละเอียดเกี่ยวกับตัวเอกสารว่าจะแสดงหรือซ่อนส่วนไหนของเอกสารบ้าง ซึ่ง DTD จะเป็นส่วนที่เพิ่มเติมสำหรับ XML ถ้าหากมีการส่งข้อมูลในรูปแบบ DTD ก็จะรู้กันว่าเป็น XML มีความหมายหลาย ๆ คำที่ อธิบายลักษณะของ XML

Richard Baldwin นิยามความหมายของ XML ไว้ดังนี้

"XML ทำให้ผู้ใช้สามารถสร้างและดูแล structured documents (เอกสารที่มีโครงสร้าง) ที่บรรจุ plain text (ตัวอักษร) โดยทำให้สามารถ rendered หรือปรับเปลี่ยนการแสดงผลในรูปแบบที่หลากหลาย จุดประสงค์หลักของ XML คือการแยกส่วน ข้อมูลเพื่อประโยชน์ในการแสดงผล"

ตามข้อความด้านบน สามารถอธิบายความหมายของคำศัพท์ ได้ดังนี้

- Structured document

พิจารณาจากตัวอย่างโครงสร้างของ หนังสือ ต่อไปนี้ดู

- o หนังสือหนึ่งเล่มประกอบจากเนื้อหาแต่ละบท (chapter)
- o ในแต่ละบทประกอบด้วยหัวข้อย่อย (section)

o ในแต่ละหัวข้อย่อ อาจจะถูกอธิบายหรือมีตารางข้อมูล (table) บรรจุอยู่

o ตารางข้อมูลถูกสร้างขึ้นมาจากแถว (row) และคอลัมน์ (column)

ดังนั้น จะเห็นว่าหนังสือแต่ละเล่ม มักจะมีรูปแบบ หรือโครงสร้างที่แน่นอน ซึ่งทุกคนที่หยิบหนังสือขึ้นมาอ่าน ก็จะเข้าใจโครงสร้างของหนังสือ ตามที่ได้อธิบายไว้ข้างต้น

- Plain text

ตัวอักษร(Character) ที่เห็นในจดหมายหรือบนกระดาษทั่วไป เป็นลักษณะตัวอักษรประเภทที่พิมพ์ได้ (alphabet and punctuation) เช่นที่เราใช้พิมพ์ในเอกสารในเวิร์ดหรือ โปรแกรมพิมพ์รายงานทั่วไป ซึ่งรวมถึงตัวเลขด้วย ถือว่าเป็น plain text ทั้งนี้และทั้งนั้น ผู้อ่านอาจจะสงสัยว่า แล้วตัวอักษรอะไรพิมพ์ไม่ได้หรือ ลองมองดูที่คีย์บอร์ดของท่าน แล้วดูที่เป็น Shift, Ctrl, Alt, Insert, Delete, ... เหล่านี้เป็นต้น

ASCII คือที่มาของรหัสตัวอักษร: รูปแบบที่ใช้แทนตัวอักษรที่นิยมใช้กันมากที่สุด เราเรียกว่า (ASCII) โดยจะแทนตัวอักษรด้วยรหัสหรือค่าตัวเลข เช่น ตัวอักษร "A" ถูกแทนด้วยค่า 65 ในขณะที่ตัวอักษร "B" แทนด้วยค่า 66 และ "C" คือ 67 ตามลำดับ รหัส ASCII นี้เอง ซึ่งถูกนำมาใช้งานในการแสดงข้อมูล รูปแบบเอกสาร ที่สามารถพิมพ์ให้ผู้ใช้ได้เห็นในขณะที่การบันทึกบนระบบคอมพิวเตอร์ จะใช้ในการเก็บเป็นตัวเลขแทน ASCII เป็นคำย่อที่มาจาก American Standards Committee on Information Interchange โดยเป็นองค์กรที่จะตั้งขึ้นมาดูแล รหัสคำสั่งต่างๆเหล่านี้ แต่ในความเป็นจริงนอกจาก ASCII จะเก็บบันทึกตัวอักษรแล้ว ยังมีรหัสที่แทนตัวอักษร ต่อไปนี้เป็นความหมายที่ทำให้เข้าใจความเกี่ยวข้องระหว่างรหัส ASCII กับ Plain text

"มาตรฐานที่เกิดจาก American Standards Committee on Information Interchange คือแนวทางที่ถูกใช้ในการกำหนดลักษณะของ plain text กล่าวคือตัวอักษรที่สามารถอ่านได้ โดยไม่ต้องใช้ความสามารถ ของโปรแกรมใดๆมาช่วยในการอ่านตัวอักษรเหล่านั้น ข้อพิเศษของ ASCII อีกอย่างหนึ่งคือ เป็นข้อมูลที่สามารถ นำไปแสดงได้ในทุกๆแพลตฟอร์ม แต่ก็มีข้อเสียที่ เป็นรูปแบบที่น่าเบื่อ (ไม่สวยงาม) ไม่สามารถแสดงในรูปแบบตัวหนา ตัวเอียง หรือขีดเส้นใต้ และไม่มีรูปแบบที่ดูเป็นกราฟิก แต่อย่างไรก็ตาม ถือว่าเป็นแบบในการแสดงตัวอักษรที่ง่ายที่สุด และเข้าใจได้ในทุก ๆ ระบบ"

ข้อมูลดิบของเอกสาร XML (Raw-XML)

เอกสาร XML ที่ถือเป็นข้อมูลดิบ คือข้อความที่เกิดจากการรวมกันของตัวอักษร เพื่อทำให้เกิดเป็นเอกสาร ก่อนที่จะนำเอกสารที่เกิดจากข้อมูลดิบ มาใช้แสดงผลลัพธ์ตามวัตถุประสงค์ของการใช้งานอีกที ซึ่งเป็นรูปแบบที่น่าสนใจมากกว่า plain text

ความหมายของการปรับการแสดงผล (Rendering)

เป็นศัพท์สำหรับคอมพิวเตอร์สมัยใหม่ rendering คือรูปแบบของการนำข้อมูลดิบ หรือ plain text มาปรับเปลี่ยนใหม่ ให้ดูน่าสนใจยิ่งขึ้น สำหรับการมองของมนุษย์ Rendering เป็นการวาดภาพ (Drawing) ตัวอย่างของการวาดภาพในระบบคอมพิวเตอร์ เป็นการนำเอาสมการ ทางคณิตศาสตร์ มาใช้ร่วมกับตัวเลข

ต่างๆ เช่น สมการวาดวงกลม กับค่าตัวเลขหนึ่งให้วงกลมขนาดหนึ่ง กับอีกตัวเลข ให้วงกลมในอีกหนึ่งขนาด ซึ่งตัวเลขและสมการเหล่านี้ เป็นสิ่งที่มนุษย์ หรือผู้ใช้งานไม่อาจได้เห็นเลย แต่จะเห็นเป็นรูปร่างที่เกิดขึ้นแทน

Rendering เอกสาร: เมื่อพูดถึงขบวนการ rendering ของเอกสาร มีความหมายว่าเป็นการแสดงข้อมูลในรูปแบบเอกสาร ที่มนุษย์เข้าใจ เช่นหนังสือ หนังสือพิมพ์ หรือเอกสารแบบอื่นๆ ที่มนุษย์เข้าใจและอ่านได้ Rendering ในที่นี้ หมายถึงการแสดงข้อมูลในรูปแบบ อักษรตัวหนา, อักษรตัวเอียง, อักษรขีดเส้นใต้, อักษรที่มีสี, เป็นต้น

แยกการแสดงผลด้วยเนื้อหา: ข้อมูลที่สร้างขึ้นเป็น XML ไม่มีความสามารถ หรือคุณลักษณะพิเศษอะไร ที่ทำให้การแสดงผลออกมาในรูปแบบที่น่าสนใจ เช่น ตัวหนา ตัวเอียง ตัวขีดเส้นใต้ ดังที่กล่าวผ่านมาแล้ว ให้จำแต่เพียงว่า XML ทำหน้าที่แยกข้อมูลออกเป็นเนื้อหา (Content) เท่านั้น ตามลักษณะเอกสารที่มีโครงสร้าง ส่วนหน้าที่การนำเอาเนื้อหาใน XML มาแสดงในรูปแบบที่น่าสนใจ เป็นหน้าที่การทำงานของส่วนอื่น

2.2.1.2 ลักษณะโครงสร้างของ XML

XML เป็นการใช้ข้อความเพื่อบ่งบอกโครงสร้างของเอกสาร พิจารณาตัวอย่างรูปแบบโครงสร้างของหนังสือ เมื่อหนังสือประกอบด้วยจำนวนบท 2 บท ในแต่ละบทประกอบด้วยเนื้อความ (Text)

Begin Book

Begin Chapter 1

Text for Chapter 1

End Chapter 1

Begin Chapter 2

Text for Chapter 2

End Chapter 2

End Book

หนังสือที่มีอยู่ในปัจจุบัน จะมีโครงสร้างที่มีรายละเอียดที่ซับซ้อนมากกว่านี้ เช่น บทนำ, สารบัญ, เป็นต้นเช่นเดียวกัน ภายในส่วนเนื้อความ (Text) ยังประกอบด้วยโครงสร้างย่อย คือ ย่อหน้า (Paragraph) แต่ละย่อหน้ายังประกอบขึ้นจาก ประโยค คำ และตัวอักษรด้วย

ลักษณะของเอกสาร XML นั้น สามารถอธิบายโดยใช้ตัวอย่างที่ 1 ได้ ดังนี้
ตัวอย่างที่ 1

```
<?xmlversion="1.0"encoding="windows-874"?>
<mali>
<malisorn>ขึ้นต้นด้วยมะลิซ้อน</malisorn>
<malila>พอแตกใบอ่อนเป็นมะลิลา</malila>
</mali>
```

```
<?xmlstylesheet type="text/xsl" href=""?>
```

บรรทัดที่ 1 นั้นหมายความว่าเราประกาศเอกสารนี้เป็นเอกสาร XML และมีการเข้ารหัสอักขระเป็น windows-874 เพื่อให้ใช้ภาษาไทยได้ จากนั้นเราจะเห็น สิ่งที่เราคุ้นเคยที่เราเรียกว่า tag แต่จริงๆแล้วใน ภาษา XML จะแบ่ง โครงสร้างเป็น 2 ส่วนใหญ่ ๆ คือ tag & element สามารถอธิบายเพิ่มเติมได้จาก ตัวอย่างที่ 2

ตัวอย่างที่ 2

```
<root>
  <element>
    <tag></tag>
  </element>
</root>
```

ความหมายของ Tag กับ Element Tag

สำหรับใน XML แล้ว tag มีความหมายในลักษณะเดียวกับที่ใช้ใน HTML tag คือข้อความที่อยู่ระหว่าง สัญลักษณ์ < และ >

- Tag เปิด (Start tag) <book>

จากตัวอย่างที่แสดง ด้านบนถูกเรียกว่า tag เปิด ดังนั้น tag เปิดจึงมีสัญลักษณ์คือ <...>

- Tag ปิด (End Tag)

</book>

tag ที่ถูกเรียกว่า tag ปิด ต่อเมื่อใน tag มีเครื่องหมาย / อยู่หลังสัญลักษณ์ < ดังนั้นลักษณะของ tag ปิดจึง มีรูปแบบคือ </...> หากพิจารณาระหว่าง tag เปิดกับ tag ปิดแล้ว ข้อแตกต่างอีกข้อหนึ่งคือ tag เปิด เป็น tag ที่สามารถใส่ข้อมูล attribute ลงไปภายใน tag ได้ แต่ tag ปิดจะไม่ทำกัน

Element

ในที่นี้คือ โครงสร้างหลักของ XML ซึ่งอยู่ในรูปของ tag เช่นเดียวกัน ตามตัวอย่างข้างบน element คือ

```
<root>
  <element>

</element>
</root>
```

จะมีลักษณะซ้อนกันเป็นชั้นๆ

- Root element เป็น element แรกสุดของเอกสาร XML

element เป็นส่วนประกอบของ Content และ attribute เพื่อให้ผู้อ่านได้เข้าใจความหมายของศัพท์หลายๆ คำศัพท์ ก่อนที่จะนำไปใช้ในการสร้าง XML ซึ่งจำเป็นอย่างมาก ลองมาพิจารณา คำจำกัดความของ element กันก่อน

```
<chap number="1">Text for Chapter 1</chap>
```

สังเกตจากลักษณะ รูปทางด้านบน ทั้งหมดตั้งแต่ <chap จนถึง </chap> ถูกเรียกว่า element หรือถ้ามองง่าย ๆ คือ element เริ่มต้นที่ tag เปิด และสิ้นสุดที่ tag ปิดใน tag คำสั่งเดียวกัน เพื่อให้ง่ายแก่การแยกแยะ สีแต่ละสีที่ให้ไว้ในตัวอย่าง แสดงความหมายที่เกี่ยวข้องกับ XML ได้ดีทีเดียว

Content

เนื้อหา หรือ Content ถือได้ว่าเป็นข้อมูลเพื่อใช้ในการแสดง ให้ผู้อ่านเอกสารได้เห็น Content คือ ตัวอักษรสีเขียว จากตัวอย่าง หรือกล่าวอีกนัยหนึ่งคือ Content อยู่หลัง tag เปิด และจบที่ก่อนถึง tag ปิดนั่นเอง

Attribute

Attribute คือตัวอักษรสีน้ำเงิน จากตัวอย่าง และจะเห็นว่า ถูกบรรจุอยู่ภายใน tag เปิด และ attribute นี้ในบางครั้ง อาจจะมีหรือไม่มีก็ได้ รวมถึงถ้ามีอาจมีได้มากกว่าหนึ่งตัวใน tag เปิด attribute คือข้อมูลความหมายเพิ่มเติมแน่นอน ถ้ากล่าวถึงสิ่งของ หากเราไม่ได้ให้ความหมายเพิ่มเติมเราก็จะไม่รู้ และแยกแยะสิ่งที่เหมือนกันออกจากกันได้ เช่น ถ้าพูดถึง บท ในหนังสือ แต่นี่เป็นความหมายโดยรวม แต่ถ้าบอกว่า บทที่ 1 ในหนังสือ เลข 1 ในที่นี้คือความหมายเพิ่มเติมให้กับบท หรืออีกซีกตัวอย่าง ถ้ากล่าวถึง แอปเปิ้ล อย่างเดียว คนทั่วไปจะนึกถึงผลไม้ที่ทานได้ แต่ถ้าบอกแอปเปิ้ล สี

แดง รสเปรี้ยว นั่นคือการบอกถึง attribute 2 ตัวคือ สีแดง และ รสเปรี้ยว ทำให้ทราบว่าเป็นแอปเปิ้ลที่ไม่มีใครอยากได้มาทาน (ยกเว้นคนที่ชอบของเปรี้ยวๆ)

2.2.1.3 วัตถุประสงค์หลักของ XML

ความหมายของ XML ก่อนหน้านี้ ได้ออกเอาไว้ว่า "จุดประสงค์หลักของ XML คือการแยกส่วนข้อมูลเพื่อประโยชน์ในการแสดงผล "นั่นคือ เอกสาร XML ใช้สำหรับควบคุม ตัวอักษร (plain text) โดยยึดเอาข้อมูลที่มีใจความเหมือนกัน แต่สามารถนำไปแสดงผล ให้ผู้ใช้หรือผู้อ่านหลายคนได้รับรู้ข้อมูลใจความเดียวกัน เมื่อมองผ่านอุปกรณ์แสดงผลที่ต่างกัน ซึ่งลักษณะเหล่านี้ ไม่เกิดขึ้นจริงกับ เอกสารประเภทประมวลผลคำอื่นๆ word processing หลากหลายคอมพิวเตอร์และหลากหลายระบบปฏิบัติการ นี่ไม่ใช่สิ่งที่ควรจะได้เลย เห็นได้ชัดว่าหลายๆปีที่ผ่านมา คอมพิวเตอร์หลากหลายชนิดถูกผลิตออกมาเป็นจำนวนมาก เพื่อใช้งาน รวมถึงระบบปฏิบัติการ (Operation System) ก็ยังมีความแตกต่างกันออกไปด้วย นั่นคือคอมพิวเตอร์รุ่นใหม่ๆจึงมีความหลากหลาย และยุ่งยากในการทำให้เข้าใจข้อมูลเดียวกัน ซึ่งเปรียบแล้วเหมือนกับภาษามนุษย์ที่ใช้สื่อสารกัน มีมากมายหลายร้อยภาษา W3C ได้กล่าวถึงจุดมุ่งหมายหลักไว้ 10 หัวข้อ ดังนี้

1. XML มีการใช้งานโดยตรงบนเครือข่าย อินเทอร์เน็ต XML จะถูกออกแบบมาสำหรับจัดเก็บ และจัดส่งข้อมูลบนเว็บ
2. XML มีการสนับสนุนโปรแกรมที่หลากหลาย ถึงแม้ว่าวัตถุประสงค์ที่สำคัญคือ การจัดส่งข้อมูลบนเว็บผ่านทางเซิร์ฟเวอร์และโปรแกรมบราวเซอร์ XML จะถูกออกแบบมาเพื่อใช้กับโปรแกรมที่มีรูปแบบต่าง ๆ ตัวอย่างเช่น การแลกเปลี่ยนข้อมูลระหว่างโปรแกรมทางการเงิน การเผยแพร่และปรับปรุงโปรแกรมให้ทันสมัยและการเขียน Voice Script ให้สื่อสารได้ด้วยโทรศัพท์
3. XML จะต้องเข้ากันได้กับ SGML
4. XML จะต้องง่ายต่อการเขียนโปรแกรมเพื่อประมวลผลเอกสาร
5. จำนวนของทางเลือกเฉพาะของ XML ควรมีจำนวนน้อยที่สุดหรือไม่ควรมีเลย
6. เอกสาร XML จะต้องอ่านเข้าใจง่ายและมีความชัดเจน
7. XML ออกแบบมาเพื่อให้พัฒนาโปรแกรมได้อย่างรวดเร็ว
8. การออกแบบ XML ต้องมีรูปแบบที่เหมาะสมและกะทัดรัด
9. สามารถสร้างเอกสาร XML ได้ง่าย
10. Markup ของ XML ต้องไม่รบกวนมากเกินไป

ภาษาสำหรับการจัดการโครงสร้างเอกสาร

XML พยายามลดข้อจำกัดของความแตกต่าง ทางด้านระบบคอมพิวเตอร์ เพื่อสร้างเป็นภาษาที่ใช้กำหนดโครงสร้างของเอกสาร ให้สามารถเข้าใจกันได้ในทุก ๆ ระบบ XML คือภาษาที่มีลักษณะเมตา (meta language)

เมตา หมายถึงรูปแบบการรวบรวมข้อมูลข่าวสาร ที่นำมาจากแหล่งข้อมูลข่าวสารอื่นๆ เช่น บทอ้างอิงในหนังสือสามารถบอกผู้อ่านได้ ถ้าถูกรวบรวมเนื้อหาไว้ด้วย XML ผู้อ่านจะสามารถรู้ว่าหัวข้อที่ตัวเองสนใจ จะไปอยู่ที่หน้าที่เท่าไรของหนังสือ ซึ่งเมื่อเปิดไปยังหน้าดังกล่าวแล้ว จะได้รับข้อมูลที่อ้างไปถึงจากหัวข้อในบทอ้างอิงนั่นเอง ดังนั้นแต่ละหัวข้อในบทอ้างอิงจึงเป็น ข้อมูล ที่ได้รวบรวมมาจากข้อมูลในเนื้อหา บทอ้างอิงจะสามารถบอกข้อมูลเกี่ยวกับ element (elements) และแอททริบิวต์ (attributes) ซึ่งจะมีเนื้อหาต่อไปได้ เราจึงใช้ XML เป็นภาษาเมตา

2.2.1.4 การประยุกต์ใช้ XML

ตัวอย่างการประยุกต์ใช้งาน XML ตัวอย่างแรก ของเอกสาร XML:

```
<book>
<chap>
Text for Chapter 1
</chap>
<chap>
Text for Chapter 2
</chap>
</book>
```

จากรูปเป็น ไวยากรณ์ที่ถูกใช้ใน XML เมื่อเปรียบเทียบตัวอย่างนี้กับเอกสารที่เห็นในส่วนต้นของบทความสำหรับโครงสร้างของหนังสือ โดยเปรียบเทียบทีละส่วน โดยพิจารณาที่ส่วนของ element (สังเกตกลุ่ม element จากสัญลักษณ์ <...> จนถึง </...>) ดังนั้นโครงสร้างของ XML จะมี element เป็นส่วนประกอบ ใช้กำหนดโครงสร้างของเอกสาร

ตัวอย่างถัดไปด้านล่าง เป็นการปรับปรุงตัวอย่าง XML ก่อนหน้าที่มี element อยู่ โดยเพิ่มส่วนที่เรียกว่า attribute ลงใน element คือตัวเลขบอกเลขที่ของบท โดยชื่อ attribute ใช้คำว่า number และ ตามด้วยตัวเลขบอกความหมายของบท

```
<book>
<chap number="1">
Text for Chapter 1
</chap>
<chap number="2">
```

Text for Chapter 2

</chap>

</book>

ตัวอย่างของคำสั่ง สำหรับการกำหนด โครงสร้างสำหรับหนังสือ

<book>

<chap number="1">

Text for Chapter 1

</chap>

<chap number="2">

Text for Chapter 2

</chap>

</book>

จากตัวอย่างโครงสร้าง เมื่อพิจารณาจะเห็นได้ว่า มีรูปแบบของจำนวนบทอยู่ 2 บท ในแต่ละบทมีข้อความอยู่บางส่วน เช่น บทที่ 1 มีข้อความ Text for Chapter 1 ที่อยู่ระหว่าง element chap จะมี attribute ชื่อ number โดยบทที่ 1 ใช้ number="1" เป็นข้อมูลของ attribute

จากโครงสร้างเอกสาร XML ที่เราได้กล่าวถึงในส่วนต้น แนนอนสามารถนำไปผ่านขบวนการ render เพื่อให้ได้เอกสารใช้งานจริงๆ ได้ในหลากหลายรูปแบบ ข้อมูลแอททริบิวต์ (attribute) มีความสำคัญ ข้อมูลอยู่ใน attribute number มีความหมายที่จะสามารถทำให้กลไก render สามารถแยกแยะเอกสารตามรายละเอียด ของเนื้อหาในแต่ละบทได้ ดังนั้นหากต้องการให้มุมมองของการมองเห็นแตกต่างกัน ก็ย่อมทำได้เช่นเดียวกัน ยกตัวอย่างเช่น หากนำเอกสาร XML ผ่านขบวนการ render สำหรับหนังสือที่เป็นกระดาษ การจัดพิมพ์ย่อจะเกิดหน้าว่างหนึ่งหน้า ของแต่ละบทเสมอ ในขณะที่เอกสารตัวเดียวกัน หากนำไปผ่านขบวนการ render ของมอนิเตอร์แล้ว ก็ไม่จำเป็นต้องทำให้เกิดหน้าว่างก่อนบท

เอกสาร XML เพียงบอกแค่ ข้อมูลในส่วนที่เป็น Content ซึ่งถูกแยกแยะข้อมูลออกเป็นส่วนๆ ในลักษณะเอกสารที่มีโครงสร้าง หรือกล่าวอีกนัยหนึ่งคือ XML ใช้สำหรับแยกแยะเอกสารออกเป็นส่วนๆ เท่านั้น โดยที่ข้อมูลในเอกสาร ไม่มีส่วนที่ใช้บอกว่าควรที่จะแสดง Content ในลักษณะการมองเห็นแบบไหน

2.2.1.5 การใช้งานเอกสาร XML

ในอนาคตคาดว่า รูปแบบการ rendering มีการพัฒนาไปอย่างมาก ดังจะเห็นได้จาก บริษัทักษ์ใหญ่ ค่ายต่าง ๆ ทางด้านไอที และคอมพิวเตอร์ ได้ร่วมมือกัน และพัฒนาโครงสร้าง หรือกลไกการ rendering ไว้ในแผนการของตนเองแล้ว หากเราสามารถนำข้อมูลของเราเอง มาเก็บไว้ในรูปแบบของ XML จะทำให้ข้อมูลของเรา มีคุณค่าในอนาคต และสอดคล้องกับเทคโนโลยีที่จะมีมาด้วย

- XML จะเกิดความสะดวกในระบบพาณิชย์อิเล็กทรอนิกส์

ตัวอย่างหนึ่งคือเทคโนโลยี XML และ Java ได้มีการกำหนดข้อตกลงต่างๆ เพื่อพัฒนาขบวนการค้า หรือ E-Commerce ในการประชุม Software Development East ที่ผ่านมา และพวกเขาเชื่อมั่นว่า ในระบบการค้าอิเล็กทรอนิกส์ ข้อมูลจะถูกเก็บอยู่ในรูปแบบของเอกสาร XML ทั้งหมด อย่างเร็วที่สุดในปลายปี 2001

- XML ทำให้ผู้ค้าและผู้ใช้มีความอิสระ

ผู้พัฒนาเทคโนโลยี มีความอิสระในการเพิ่มศักยภาพในผลิตภัณฑ์ของตนเอง ไม่จำเป็นต้องมากังวลรูปแบบการสื่อสาร ที่จะต้องออกแบบมาเฉพาะ ทำให้ลูกค้าต้องยึดติดกับผลิตภัณฑ์รายใดรายหนึ่ง เมื่อข้อมูลอยู่ในเอกสาร XML แล้ว ผลิตภัณฑ์ต่างๆ ก็จะมีรูปแบบข้อมูลที่สามารถแลกเปลี่ยนกันได้ โดยที่ไม่ต้องไปกังวลกับเทคโนโลยี ที่ไม่สอดคล้องกันในปัจจุบันได้

- XML ลดค่าใช้จ่าย

แน่นอน ผลที่ตามมา กับ เทคโนโลยีที่แลกเปลี่ยนข้อมูลกันได้ ทำให้ค่าใช้จ่ายที่จะต้องสูญเสียไป ในปัจจุบันสำหรับการสื่อสารข้อมูล ที่มีรูปแบบที่หลากหลายถูกขจัดออกไป นั้นเป็นผลดีสำหรับยุคการสื่อสารด้วย XML

2.2.1.6 XSLT และ Xpath

XSLT เป็นภาษาที่สามารถแปลงเอกสาร XML ให้เป็นเอกสารแบบเท็กซ์ เป็น XML เองหรือเป็นฟอร์แมตอื่นๆ ได้ โดย XSLT เป็นส่วนประกอบย่อยของภาษาที่ใหญ่กว่าที่เรียกว่า XSL ซึ่ง XSLT เป็นเครื่องมือที่มีศักยภาพสำหรับการค้าอิเล็กทรอนิกส์และที่อื่นใดที่ XML จะถูกนำมาใช้ได้

ยกตัวอย่างเช่น ถ้าเรากำลังรันเว็บไซต์เกี่ยวกับข่าวที่นอกเหนือจากเว็บไซต์ของเราเองแล้ว เรายังต้องการให้ผู้อ่านมีหนทางที่จะได้รับข่าวของเราในรูปแบบ XML ด้วย หรืออีกนัยหนึ่งก็คือ คนทั่วไปสามารถนำข่าวที่เราจัดหาไว้ไปใช้กับแอปพลิเคชันของพวกเขาเองได้ ไม่ว่าด้วยวัตถุประสงค์อะไรก็ตาม อย่างเช่น บางคนอาจจะต้องการแสดงเรื่องราวดังกล่าวบนเว็บไซต์ของพวกเขาเอง แต่คนอื่นๆ ก็อาจจะต้องการใส่ข้อมูลที่ได้นำเข้าไปในฐานข้อมูลของพวกเขาได้

นี่เป็นที่มาของ XSLT กล่าวคือ แทนที่ที่คนอื่นได้รับเอกสาร XML ของเราไป เราสามารถที่จะใช้ภาษาในการแปลงรูปแบบเพื่อแปลงเอกสารเหล่านี้ไปเป็นรูปแบบอะไรก็ตามที่พวกเขาต้องการได้ ไม่ว่าจะเป็น HTML เพื่อนำไปแสดงผลบนเว็บไซต์ เป็น XML ที่มีโครงสร้างต่างไปจากเดิมเพื่อใช้กับแอปพลิเคชันอื่นๆ หรือแม้แต่เป็นเท็กซ์ไฟล์ธรรมดาาก็ได้

2.2.1.6.1 XSLT

Extensible Style sheet Language for Transformation หรือ XSLT คือภาษาที่สามารถแปลงเอกสาร XML ให้เป็นเอกสารแบบ เท็กซ์, เป็น XML เองหรือเป็นฟอร์แมตอื่นๆ ได้

XSLT เป็นส่วนประกอบย่อยของภาษาที่ใหญ่กว่าที่เรียกว่า XSL ซึ่ง XSLT เป็นเครื่องมือที่มีศักยภาพสำหรับการค้าอิเล็กทรอนิกส์และที่อื่นใดที่ XML จะถูกนำมาใช้ได้ XSLT จะทำงานด้วยการค้นหาส่วนของเอกสาร XML ที่ตรงกับชุดของเท็มเพลตที่กำหนดเอาไว้แล้ว หลังจากนั้นก็จะประยุกต์ใช้ในการแปลงข้อมูล และการจัดรูปแบบเข้ากับส่วนที่ค้นพบภาษาอื่นๆ เช่น XPath ถูกนำมาใช้เพื่อช่วยในการค้นหาส่วนที่เหมาะสม

2.2.1.6.2 XSL คืออะไร

Extensible Style sheet Language มีความหมายตามชื่อของมัน นั่นคือ ภาษาที่มีลักษณะแบบ XML (XML-based) ที่ใช้ในการสร้างสไตล์ชีต กลไกของ XSL ใช้สไตล์ชีตเพื่อแปลงเอกสารเพื่อแปลงเอกสาร XML ทั้งหมดไปเป็นเอกสารชนิดอื่นๆ และเพื่อจัดรูปแบบของเอาต์พุตเหล่านี้ คุณจะกำหนดเลย์เอาต์ของเอกสารเอาไว้ รวมทั้งที่ได้อ้างอิงมาใช้ภายในเอกสารอินพุต ก็เหมือนกับการพูดว่า “ข้อมูลของคุณอยู่ ณ จุดใดในเอกสารอินพุต ให้ทำให้เป็นแบบนี้ในเอกสารเอาต์พุตนะ” นั่นเอง ภาษาเฉพาะของ XSL จะเรียกเอกสารอินพุตว่าต้นไม้ต้นฉบับ (source tree) ส่วนเอกสารเอาต์พุตเรียกว่า ต้นไม้ผลลัพธ์ (result tree)

จริงๆ แล้วมีภาษาที่สมบูรณ์แล้วอยู่ 2 ภาษาภายใต้ร่มเงาของ XSL ได้แก่ :

- ภาษาสำหรับแปลงข้อมูลที่มีชื่อว่า XSLT
- ภาษาที่ใช้ในการอธิบายเอกสาร XML เพื่อการแสดงผล นั่นคือ XSL Formatting Object

เป็นที่แน่นอนว่าทั้งสองภาษานี้สามารถใช้เพื่อการแปลงข้อมูลของ XSLT ร่วมกันได้ และ

ออกแบบเจตน์ในการจัดรูปแบบของ XSL จะทำการปรับปรุงข้อมูลเพื่อการแสดงผลต่อไป

นอกจาก XML ที่ลงตัวแล้ว XSLT ยังให้ผลลัพธ์ที่เป็น HTML หรือแม้แต่เท็กซ์ธรรมดาาก็ได้ด้วย ในความเป็นจริงนั้น หนึ่งในการใช้งานที่เน้นมากๆ สำหรับ XSLT ก็คือ ใช้เพื่อแปลงเอกสาร XML ไปเป็นเอกสาร HTML เพื่อแสดงในเบราว์เซอร์

2.2.1.6.3 สไตล์ชีต XSLT ทำงานอย่างไร

XSLT สไตล์ชีตถูกสร้างขึ้นมาจากโครงสร้างที่เรียกว่า เท็มเพลต (Template) เท็มเพลตจะเป็นตัวกำหนดสิ่งที่ต้องค้นหาจากต้นไม้ต้นฉบับ และสิ่งที่ต้องใส่เข้าไปในต้นไม้ผลลัพธ์

XSLT เขียนขึ้นด้วย XML หมายความว่า มีอิลิเมนต์และ แอททริบิวต์พิเศษของ XSLT ที่คุณใช้เพื่อสร้าง สไตร์ชีตของคุณเองได้ดังตัวอย่างต่อไปนี้ :

```
<xsl : template match="first"> My Project </xsl : template>
```

ใน XSLT เพิ่มเพลตถูกกำหนดขึ้นโดยใช้อิลิเมนต์ `<xsl : template>` ที่มี 2 ส่วนสำคัญคือ แอททริบิวต์ `match` และเนื้อหาของเพิ่มเพลต แอททริบิวต์ `match` จะกำหนดรูปแบบในต้นไม้ต้นฉบับดังในตัวอย่างข้างต้น จะใช้เพิ่มเพลตนี้กับทุกโหนดในต้นไม้ต้นฉบับที่จับคู่กับรูปแบบนั้นๆ ซึ่งในกรณีนี้ก็จะใช้เพิ่มเพลตนี้กับทุกอิลิเมนต์ที่มีชื่อว่า `first`

คุณได้ระบุสิ่งที่จะได้ออกมาเป็นต้นไม้ผลลัพธ์ไว้ในเนื้อหาของอิลิเมนต์ `template` ในที่นี้เราได้กำหนดข้อความบางอย่างขึ้นมามางๆ ถ้าเพิ่มเพลตนี้ถูกรวมเข้าไปในสไตร์ชีตแล้วทุกๆ ครั้งที่กลไก XSLT ค้นพบอิลิเมนต์ `<first>` ในต้นไม้ต้นฉบับมันก็จะให้เอาต์พุตออกมาเป็นข้อความว่า “My Project”

เนื้อหาของเพิ่มเพลตสามารถเป็นอะไรที่สลับซับซ้อนมากกว่าการให้เอาต์พุตเป็นข้อความธรรมดา และก็เป็นอย่างนั้นเสมอ มีอิลิเมนต์ของ XSLT อยู่จำนวนหนึ่งที่คุณสามารถใส่เข้าไปในเพิ่มเพลตเพื่อให้ทำหน้าที่หลากหลายอย่างได้ ยกตัวอย่างเช่นมีอิลิเมนต์หนึ่งคือ `<xsl : value-of>` ที่สามารถเอาข้อมูลจากต้นไม้ต้นฉบับแล้วเพิ่มมันเข้าไปในต้นไม้ผลลัพธ์ได้ โค้ดต่อไปนี้จะทำงานเช่นเดียวกันกับเพิ่มเพลตก่อนหน้านี้ของเรา แต่แทนที่จะให้เอาต์พุตเป็น “My Project” เพิ่มเพลตนี้จะเพิ่มเนื้อหาของอิลิเมนต์ใดๆ ที่มีชื่อว่า `first` เข้าไปในต้นไม้ผลลัพธ์ :

```
<xsl : template match="first"> <xsl : value-of select = "." / ></xsl : template>
```

พูดง่ายๆ คือ ถ้ากลไกการทำงานของ XSLT ค้นพบ :

```
<first> Anuruk </first>
```

มันจะแสดงเอาต์พุต Anuruk ออกมา และถ้าค้นพบ :

```
<first> Anuchin </first>
```

ก็จะแสดงเอาต์พุต Anuchin ออกมา

การเชื่อมความสัมพันธ์ระหว่างสไตร์ชีตด้วยคำสั่งประมวลผล

XSL สไตร์ชีตสามารถเชื่อมความสัมพันธ์กับเอกสาร XML ด้วยการใส่คำสั่งประมวลผล (PI) ที่เป็นชนิดเดียวกันกับสไตร์ชีต ดังนี้

```
<?xml – stylesheet type = “text / xsl” href = “stylesheet.xml” ?>
```

อย่างไรก็ตาม เนื่องจากปัจจุบัน IE5 เป็นเว็บเบราว์เซอร์เดียวที่เข้าใจภาษา XML และ XSL ดังนั้นสไตร์ชีตของคุณจะต้องใช้ XSLT ที่เป็นไวยากรณ์เก่า เพื่อให้กลไกการทำงานของ XSLT ของ IE5 เข้าใจได้อย่างถูกต้อง

2.2.1.6.4 XPath

สำหรับภาษาที่มีชื่อเรียกว่า XPath ซึ่งทำหน้าที่ในการชี้ตำแหน่ง (address) ของส่วนต่างๆ ของเอกสาร XML เพื่อที่จะอนุญาตให้เราได้รับชิ้นข้อมูลที่เราต้องการได้ XSL ใช้ XPath อย่างแพร่หลาย คุณสามารถใช้ XPath เพื่อที่จะบอกตำแหน่งเอกสารของคุณได้โดยการกำหนดตำแหน่งท้องถิ่น (location path) ซึ่งจะนำพา XPath ว่าคุณกำลังจะไปที่ไหนในเอกสารทีละขั้นๆ แน่นอนว่า, คุณไม่สามารถรู้ว่าคุณจะไปที่ไหนได้จนกว่าที่คุณจะรู้ว่าคุณอยู่ที่ใดเสียก่อน ดังนั้น XPath จึงจำเป็นต้องรู้จัก context node ซึ่งก็คือ ส่วนของเอกสาร XML ที่คุณกำลังจะตั้งต้นอยู่นั่นเอง (เราได้เห็นทางลัดใน XPath สำหรับ context node เริ่มต้น ซึ่งก็คือ “.” อย่างง่ายๆ)

ให้คิดว่าคุณภาพของ XPath ก็คือการบอกทิศทางตลอดเอกสาร XML นั่นเอง (เช่น “ตอนนี้คุณอยู่ตรงไหน คุณต้องลงไปจากตรงนี้ แล้วเลี้ยวขวาตรงถนนถัดไป ก็จะเห็นบ้านหลังแรกอยู่ทางซ้ายมือ ฯลฯ”)

เอกสาร XML สามารถมองเป็นเหมือนต้นไม้ที่มีกิ่งและใบไม้ แต่ไม่ใช่แค่โหนดเท่านั้นที่ประกอบอยู่ในต้นไม้ดังกล่าว ยังมีแอททริบิวต์, คอมเมนต์, คำสั่งประมวลผล และสิ่งอื่นใดก็ตามที่อยู่ในเอกสารก็สามารถเป็นส่วนหนึ่งของต้นไม้ที่วันนี้ได้ด้วย ด้วยเหตุผลนี้มันจึงมีประโยชน์ที่จะทำงานกับความสัมพันธ์แบบแม่/ลูกของเอกสาร XML หนึ่งๆ โดยไม่ต้องไปกังวลว่ากิ่งและใบจะเป็นโหนด, แอททริบิวต์, ชิ้นส่วนของเท็กซ์หรืออะไรก็ตาม

XPath ใช้โหนดเงื่อนไข (term node) เพื่ออ้างถึงส่วนต่างๆ ของโหนด ไม่ว่าจะเป็นโหนด, แอททริบิวต์ หรืออะไรก็ตาม

ต่อไปจะพูดถึงโหนดเซต (node-set) ซึ่งเป็นคอลเลกชันของโหนด ยกตัวอย่างเช่น ถ้าคุณบอกให้ XPath ค้นหาโหนดใดๆ ที่มีแอททริบิวต์ id อยู่ มันก็อาจจะมามากกว่าหนึ่งโหนดก็ได้ XPath ก็จะคืนค่ามาเป็นคอลเลกชันของโหนดทั้งหมดในต้นไม้ต้นฉบับที่มีแอททริบิวต์ id อยู่ โหนดเซตสามารถบรรจุโหนดชนิดใดๆ ก็ได้ ไม่ใช่เพียงแค่โหนด

2.2.1.6.5 พาทบอกตำแหน่ง

สิ่งแรกที่เราควรสังเกตเกี่ยวกับ XPath ก็คือ คอนเซ็ปต์ของรากของเอกสาร (document root) ซึ่งไม่ใช่โหนดราก เนื่องจากมันสามารถเป็นสิ่งที่อื่นใดก็ได้ที่อยู่ตอนต้นหรือตอนท้ายของเอกสาร XML อยู่ก่อนหรืออยู่หลังโหนดราก XPath ต้องการรากของเอกสารเสมือน (virtual document root) ที่ทำหน้าที่เป็นรากของลำดับชั้นของเอกสาร โดยข้อเท็จจริงแล้ว โหนดรากถือเป็นลูกของรากของเอกสารนั่นเอง

รากของเอกสารไม่ได้ชี้เฉพาะเจาะจงไปที่โหนดใดๆ ในเอกสาร เป็นเพียงแค่ว่า (root) ตามคอนเซ็ปต์ของเอกสารเท่านั้น โดยใน XPath นั้น รากของเอกสารจะถูกระบุด้วย “/”

2.2.1.7 DOM

ตอนนี้เรามีข้อมูลของเราในรูปแบบ XML แล้ว เราสามารถที่จะทำอะไรกับข้อมูลดังกล่าวในแอปพลิเคชันที่เราเขียนขึ้นได้ เราไม่ต้องการแค่เข้าถึงมันได้เท่านั้น แต่ต้องการแก้ไขและเพิ่มเติมมันได้อีกด้วย จึงเกิดเป็น Document Object Model (DOM) ขึ้น เพื่อให้ทางสายกลางในการทำงานกับเอกสาร XML ทั้งหมด (และเอกสารประเภทอื่นๆ) ผ่านการใช้โค้ดและยังใช้วิธีการอินเตอร์เฟซกับโค้ดดังกล่าวในโปรแกรมที่เราเขียนขึ้นด้วย

ยกตัวอย่างเช่น DOM จะทำให้เราสร้างเอกสารและส่วนต่างๆ ของเอกสาร, ท่องไปตามเอกสาร, ย้าย, สำเนา และลบส่วนต่างๆ ของเอกสาร, เพิ่มหรือแก้ไขแอททริบิวต์ได้

2.2.1.7.1 DOM คืออะไร

เอกสาร XML หนึ่งๆ ถูกกำหนดโครงสร้างขึ้นมากล้ายกับโมเดลเชิงวัตถุเป็นอันมาก มันเป็นลำดับชั้นที่มีโหนดอื่นเป็นลูกอย่างมีประสิทธิภาพ

สังเกตว่า โมเดลเชิงวัตถุจะสามารถทำงานได้กับเอกสารนั้นๆ เท่านั้น การทำงานกับเอกสารชนิดอื่นๆ เราจะต้องสร้างโมเดลเชิงวัตถุใหม่ๆ ให้กับมัน ในขณะที่โมเดลเชิงวัตถุเฉพาะเหล่านี้จะมีประโยชน์ในหลายๆ สถานการณ์ แต่ก็ยังมีอีกหลายสถานการณ์ที่วิธีการจัดการต่างๆ ไปก็เป็นสิ่งจำเป็น นั่นหมายความว่า เราต้องการโมเดลเชิงวัตถุที่สามารถจัดรูปแบบของเอกสาร XML ใดๆ ไม่ว่าจะถูกจัดโครงสร้างมาอย่างไร Document Object Model (DOM) จะจัดการเรื่องทั่วไปพวกนี้ให้เรา

DOM มักจะถูกใส่เข้าไปเป็นชั้นที่คั่นระหว่าง XML parser กับแอปพลิเคชันที่ต้องการข้อมูลในเอกสาร นั่นหมายความว่า parser จะอ่านข้อมูลจากเอกสาร XML แล้วป้อนข้อมูลนั้นให้กับ DOM จากนั้น DOM ก็จะถูกใช้โดยแอปพลิเคชันระดับที่สูงกว่าแอปพลิเคชันสามารถทำอะไรที่มันต้องการกับข้อมูลดังกล่าวก็ได้ รวมถึงการใส่มันเข้าไปในโมเดลเชิงวัตถุอื่นๆ ถ้ามันต้องการเช่นนั้น

ดังนั้นการที่จะเขียนแอปพลิเคชันที่จะเข้าถึงเอกสาร XML ผ่าน DOM ได้ คุณจะต้องมี XML parser และการอิมพลีเมนต์ DOM ติดตั้งเอาไว้ในเครื่องของคุณ แต่ในตอนนี้เราจะให้ความสำคัญกับการใช้ DOM ก่อนการอิมพลีเมนต์ DOM บางอย่าง

2.2.1.7.2 DOMString

เพื่อที่จะมั่นใจได้ว่าการอิมพลีเมนต์ DOM ทั้งหมดนั้นจะทำงานได้เหมือนกัน DOM จึงได้กำหนดชนิดข้อมูล DOMString ขึ้นมา ซึ่งเป็นลำดับของหน่วย (ตัวอักษร) ขนาด 16 บิต ที่จะถูกใช้ในที่ใดๆ ที่ต้องการใช้สตริง พุดอีกอย่างหนึ่งก็คือ DOM ได้ระบุเอาไว้ว่า สตริงทั้งหมดต้องเป็น UTF-16 เท่านั้น ถึงแม้ว่าข้อกำหนดของ DOM จะใช้ชนิดข้อมูล DOMString ในทุกๆ ที่ที่มีการพุดถึงสตริง แต่จริงๆ แล้วสิ่งนี้มีไว้เพื่ออำนวยความสะดวกเท่านั้น การอิมพลีเมนต์ DOM หนึ่งๆ ไม่จำเป็นต้องสร้างออบเจกต์ DOMString ประเภทใดๆ ขึ้นมาใช้เลย

อินเทอร์เน็ตเฟรมเป็นคอลเลกชันของเมธอดและพร็อพเพอร์ตี้ที่ออกแบบเจ็ทหนึ่งๆ หรือมากกว่านั้นให้การสนับสนุน

2.2.1.7.3 การใช้งาน DOM

Document Object Model (DOM) ไม่ได้เป็นเพียง API ที่ทำงานร่วมกับเอกสาร XML เท่านั้น แต่สามารถใช้เพื่อทำงานร่วมกับเอกสาร HTML, สไตล์ชีต CSS และเอกสารอื่นๆ ได้ด้วย โดยความเป็นจริงการนำ DOM ไปใช้สามารถกำหนดขอบเขตให้ทำงานเฉพาะกับเอกสาร XML หรือเฉพาะกับเอกสาร HTML หรือสามารถถูกสร้างให้ทำงานกับเอกสารชนิดต่างๆ ได้ด้วย เช่น DOM ที่เป็นส่วนหนึ่งของ parser XML จะมีเพียงการใช้ API ที่เฉพาะเจาะจงสำหรับ XML เท่านั้น เพราะเหตุว่า DOM ที่เป็นส่วนหนึ่งของเว็บเบราว์เซอร์จะมี API ที่เฉพาะเจาะจงสำหรับ HTML และ CSS รวมอยู่ด้วย เพื่อให้เข้าถึงข้อมูลชนิดเหล่านี้ทางโปรแกรมได้

การรวมกันของภาษาสคริปต์ DOM (ด้วย API สำหรับ HTML) กับ HTML คือสิ่งที่ทำให้ Dynamic HTML (หรือ DHTML) มีความเป็นไปได้ เนื่องจากเนื้อหาของเอกสาร HTML นั้นจะถูกนำมาแสดงผ่านโมเดลเชิงวัตถุเมื่อเว็บเพจหนึ่งๆ ถูกโหลดเข้าไปในเว็บเบราว์เซอร์ที่รับรู้ DHTML ออบเจกต์ทั้งหลายก็จะถูกสร้างขึ้นให้กับแต่ละอิลิเมนต์และทุกๆ อิลิเมนต์บนเพจ สิ่งนี้ทำให้คนที่เขียนเว็บเพจได้สคริปต์ลงไปในเพจได้เพื่อเรียกใช้เมธอดและพร็อพเพอร์ตี้ทั้งหลายของออบเจกต์เหล่านั้น

เนื่องจากมีประเภทของการนำ DOM ไปใช้ต่างๆ กัน DOM จึงจัดให้มีแกนของ DOM (DOM Core) ซึ่งเป็นเซตของอินเทอร์เน็ตเฟรมที่เป็นแกนหลักในการทำงานกับเอกสารขั้นพื้นฐานรวมถึงโมดูลทั้งหลายที่มีให้เลือกจำนวนหนึ่งเพื่อทำงานกับเอกสารอื่นๆ เช่น DOM HTML, DOM CSS เป็นต้น โมดูลเหล่านี้เป็นเซตของอินเทอร์เน็ตเฟรมเพิ่มเติมที่สามารถนำไปใช้ได้

อินเทอร์เน็ตเฟรมที่เป็นแกนหลักเหล่านี้สามารถแบ่งออกได้เป็นอินเทอร์เน็ตเฟรมมูลฐาน (Fundamental Interface) และ อินเทอร์เน็ตเฟรมขยาย (Extended Interface)

- อินเทอร์เน็ตเฟรมมูลฐานต้องถูกใช้งานโดยการทำงานของ DOM เท่านั้น แม้กระทั่งอินเทอร์เน็ตเฟรมที่ทำงานได้กับ เอกสารแบบ non-XML เท่านั้นก็ตาม
- อินเทอร์เน็ตเฟรมส่วนขยายต้องถูกใช้งานโดยการทำงานของ DOM เท่านั้น ซึ่งจะทำงานกับ HTML

2.2.1.8 ประโยชน์จาก XML

สำหรับประโยชน์ของ XML นั้น เป็นด้านความยืดหยุ่นในการใช้งานสำหรับแอปพลิเคชันที่อิงกับ Web Base ที่ใช้ง่ายในการค้นหาข้อมูล มีความยืดหยุ่นในการพัฒนาเว็บ สามารถผสมผสานข้อมูลจากหลายแหล่ง จากแอปพลิเคชันที่ต่างกัน สามารถแสดงข้อมูลแบบต่างๆ และสามารถ update ข้อมูลให้ทันสมัยเสมอ และคาดว่าจะเป็มาตรฐานใหม่ของระบบเปิด ซึ่งนับเป็น format ใหม่สำหรับการส่งข้อมูลบนเว็บที่มากด้วยข้อมูลหลายแบบ แต่ส่งผ่านด้วยเทคโนโลยีที่บีบอัดข้อมูลที่ให้ความเร็วได้รับการสนับสนุนจากผลิตภัณฑ์ค่ายไมโครซอฟท์

สถาปัตยกรรม XML

ภาษา XML ได้รับการสนับสนุนจาก W3C ให้นักพัฒนาเว็บแอปพลิเคชันได้หันมาใช้เป็นส่วนประกอบของการพัฒนาเว็บไซต์ เพราะ XML มีประสิทธิภาพและมีความน่าเชื่อถือสูงในการแปลงข้อมูลและโครงสร้างข้อมูลให้สามารถนำไปใช้งาน สถาปัตยกรรม 3 Tiered ที่ XML สามารถสร้างขึ้นจากระบบข้อมูลที่ใช้โมเดลของ 3-tier โครงสร้าง ของข้อมูลต่างๆ สามารถนำมาแสดงตามข้อกำหนด หรือรูปแบบที่ต้องการตามการใช้งานได้ (ดูภาพประกอบ) เราจะเห็นโครงสร้างทางสถาปัตยกรรมของ XML ที่ยืดหยุ่นเหมาะกับการพัฒนาร่วมกับเว็บแอปพลิเคชัน

ส่วนประกอบของข้อมูล

XML เป็นการทำงานในระดับกลาง middle tier ที่สามารถเรียกใช้งานข้อมูลได้หลากหลายระบบฐานข้อมูลและโอนข้อมูลให้อยู่ในรูปแบบของ XML และมีการให้รายละเอียดเกี่ยวกับตัวข้อมูล โครงสร้างต่างๆ ของระบบฐานข้อมูลได้ XML เป็นระบบเปิดที่นำเสนอข้อมูลในรูปแบบ text ผ่านทาง HTTP เหมือนกับ HTML แต่จะมีคุณสมบัติในการให้ข้อมูลแบบ real time อัปเดตหรือเปลี่ยนแปลงได้ตามข้อกำหนด การแสดงข้อมูลจาก XML ใน HTML จะเป็นการเพิ่มในส่วนขงรายละเอียดข้อมูล ที่มีการเรียกใช้จากแหล่งหรือฐานข้อมูลที่เกี่ยวข้องกันในหลายแหล่ง เพื่อให้ HTML มีความสมบูรณ์มากขึ้น ในอนาคตการพัฒนาเว็บหรือการเขียนและสร้าง HTML ไม่จำเป็นต้องมีการเขียนชุดคำสั่งที่ยุ่งยาก ซับซ้อนมากก็สามารถทำงานร่วมกับระบบข้อมูลได้อย่างมีประสิทธิภาพ XML จะทำการกำหนดค่าสำหรับโครงสร้างข้อมูลที่จะนำไปแสดงใน HTML นอกจากนั้นยังสามารถนำไปสนับสนุนระบบการแลกเปลี่ยนข้อมูลข่าวสารทาง Electronic ได้อย่างดีอีกด้วย

เครื่องมือและการสนับสนุน

เครื่องมือที่่ใช้ในการสนับสนุนการพัฒนาระบบที่ใช้มาตรฐาน XML ได้รับการสนับสนุนจากคู่ค้าหลายราย นอกจากนั้น ได้มีการร่วมมือเพื่อกำหนดมาตรฐานในการพัฒนาแอปพลิเคชันให้รองรับการทำงาน ไม่ว่าจะเป็นทางด้านระบบข้อมูล ระบบสารสนเทศ ข่าวสารที่ต้องการความรวดเร็ว เพื่อทันต่อเวลาการนำเสนอ

2.2.1.9 บทสรุปของ XML

XML มีความยืดหยุ่นพอในการนำเสนอข้อมูลแบบต่างๆ ที่สามารถให้รายละเอียดโครงสร้างข้อมูลตามระดับและความต้องการในการนำไปใช้งาน

2.3 ASP.NET เว็บแอปพลิเคชัน

2.3.1 ที่มาของ ASP

แอคทีฟเซิร์ฟเวอร์เพจ(ASP) ของบริษัทไมโครซอฟท์ คือเทคโนโลยีใหม่ในการสร้างเว็บเพจแบบไดนามิก หน้าของ ASP นั้น ประกอบด้วย HTML markup และสคริปต์ทางฝั่งเซิร์ฟเวอร์ ซึ่งจะประมวลผลตาม อินพุตที่ได้รับเข้ามา ณ ขณะนั้น และสร้างโค้ด HTML ขึ้นมาจากการประมวลผล สคริปต์ทางฝั่งเซิร์ฟเวอร์จะทำงานเมื่อมีการร้องขอ (request) หน้า ASP ส่งมาถึงฝั่งเซิร์ฟเวอร์ อินพุตของหน้า ASP จะมาจากบราวเซอร์ทางฝั่งไคลเอนต์ผ่านทางเมธอด HTTP POST และ GET ASP มี อ็อบเจกต์โมเดลเพื่อความสะดวกขึ้นในการทำงานของผู้พัฒนา นอกจากการใช้อ็อบเจกต์จาก ASP อ็อบเจกต์โมเดล เช่น Application , Server , Request , Respond และ Session แล้ว ผู้พัฒนายังสามารถใช้ COM คอมโพเนนต์ อื่นๆ ที่มีอยู่บนเซิร์ฟเวอร์ ได้

ในการพัฒนาแอปพลิเคชัน โดยใช้ ASP มักจะมีความยุ่งยากระหว่าง HTML markup และสคริปต์ทางฝั่งเซิร์ฟเวอร์ รูปแบบของหน้า ASP ทำให้เกิดความยุ่งยากในการจัดการและการนำมาใช้ใหม่ จึงมีการพัฒนาโดยเพิ่มไฟล์และฟังก์ชันในสคริปต์ทางฝั่งเซิร์ฟเวอร์ อย่างไรก็ตาม ความพยายามนี้ทำให้เกิดข้อเสียในด้านเวลา การจัดการเครือข่ายของไฟล์ที่เกี่ยวข้อง และยังมีผลกระทบทางด้านประสิทธิภาพจากการมี include ซ้อนกัน และการจัดการอ็อบเจกต์ ไลต์ และ ขอบเขตของตัวแปร

ผู้พัฒนาที่ต้องทำงานที่เกี่ยวข้องกับการทำงานกับบราวเซอร์หลายตัวมักพบปัญหาในการทำเว็บแอปพลิเคชันให้สามารถทำงานได้กับหลายบราวเซอร์ซึ่งจะต้องคำนึงถึงความสามารถของบราวเซอร์ที่ไคลเอนต์ใช้ ซึ่งวิธีการแก้ไขปัญหามักจบลงด้วยการใช้แท็กHTMLที่เป็นแท็กพื้นฐานที่สุด และใช้สคริปต์ทางฝั่งไคลเอนต์เพื่อให้เป็นที่เข้าใจได้โดยทุกบราวเซอร์บางครั้งมีความพยายามสร้างHTML marker สำหรับแต่ละบราวเซอร์โดยเฉพาะ ซึ่งได้ผลดีแต่เสียเวลาและต้องใช้ความพยายามอย่างมาก

การเขียนสคริปต์ใน ASP ใช้ภาษาได้จำกัดเช่น VB สคริปต์ และ Java สคริปต์เป็นต้น และสคริปต์ ทางฝั่งเซิร์ฟเวอร์ จะถูกแปลทุกครั้งที่หน้าASPนั้นถูกเรียก ซึ่งไม่เป็นผลดีต่อประสิทธิภาพในการทำงาน นอกจากนั้นในเรื่องการจัดการกับสถานะ ใน ASP นักพัฒนาจะต้องทำทุกอย่างด้วยตนเอง โดยจะต้องเก็บ สถานะไว้ใน SQL เซิร์ฟเวอร์ หรือเนื้อที่เก็บข้อมูลภายนอกอื่นๆ จะเห็นได้ว่า

จะเห็นได้ว่าแม้ASP จะเป็นเทคโนโลยีที่ดีในการสร้างเว็บเพจแบบไดนามิก แต่ก็ยังมีปัญหาอยู่อีกหลายด้าน จึงมีการแก้ไขและพัฒนามาเป็น ASP.NET

2.3.2 ที่มาของ ASP.NET

นักพัฒนาที่ใช้ Visual Basic มักจะคุ้นเคยกับความสะดวกสบายในการเขียนโปรแกรมโดยใช้ฟอร์ม และคอนโทรลซึ่งมีรูปแบบที่สะดวก โดยลากสิ่งที่ต้องการลงมาไว้บนฟอร์มและเขียนโค้ดโดยใช้การเขียนโปรแกรมแบบ event-handling เนื่องจากวิธีนี้เป็นวิธีที่ไม่ยุ่งยาก จึงได้ถูกนำมาใช้ใน ASP.NET

ASP.NET ทำให้การพัฒนาหน้าเว็บสะดวกขึ้นด้วยการเขียนโปรแกรมโดยใช้ฟอร์ม ใน ASP.NET เราเรียกฟอร์มเหล่านี้ว่า เว็บฟอร์ม (Web Forms) ซึ่งนำมาแทนที่หน้า ASP การเขียนโปรแกรมแบบเว็บฟอร์มนี้เป็นแบบ event base เช่นเดียวกับใน visual basic นอกจากนี้ ASP.NET ยังแยกส่วนระหว่างส่วนแอปพลิเคชันและส่วนในการแสดงผลอีกด้วย

ASP.NET เกี่ยวข้องกับรูปแบบการเขียนโปรแกรม ASP โดยมีคุณสมบัติพิเศษเพิ่มเติมขึ้นมา ดังนี้

- มีการแยกส่วนเค็ดขาดระหว่างส่วนโค้ดการประมวลผลของแอปพลิเคชันและส่วนแสดงผล (HTML markup) จึงไม่มีความสับสนในโค้ดทั้งสองส่วนอีก
- มีกลุ่มของเซิร์ฟเวอร์คอนโทรลจำนวนมากให้เลือกใช้ ซึ่งจะถูกละเปลี่ยนเป็น HTML โดยอัตโนมัติ
- มีการจัดการสถานะโดยใช้ฮอ็บเจ็คต์เซสชัน
- เป็นรูปแบบการเขียนโปรแกรมแบบ event-base ทางฝั่งเซิร์ฟเวอร์ ทำให้ง่ายขึ้น
- สามารถเขียนโค้ดส่วนแอปพลิเคชันโดยใช้ภาษาใดใน Microsoft.NET ก็ได้ เช่น ภาษา VB, C# เป็นต้น โค้ดของแอปพลิเคชันบนฝั่งเซิร์ฟเวอร์จะถูกคอมไพล์เพื่อประสิทธิภาพการทำงานที่ดีกว่า
- มี Visual Studio.NET เป็นเครื่องมืออำนวยความสะดวกในการพัฒนาเว็บฟอร์ม

2.3.3 รู้จักกับ System.Web.UI Namespace

ในส่วนนี้จะอธิบายถึงคลาส ที่สำคัญที่อยู่ในเนมสเปซชื่อ System.Web.UI ในเฟรมเวิร์คของ ASP.NET เนมสเปซ System.Web.UI นั้นระบุคลาสและอินเตอร์เฟซที่ใช้ในการสร้างสิ่งต่างๆ ในคลาสเว็บฟอร์ม ที่สำคัญที่สุดใน System.Web.UI คือคลาส Control ซึ่งบอกคุณสมบัติ เมธอด และ event ของแต่ละเซิร์ฟเวอร์คอนโทรลในเฟรมเวิร์คของเว็บฟอร์ม อีกคลาสที่สำคัญคือเพจ ซึ่งจะกล่าวถึงในลำดับต่อไป

2.3.3.1 คลาส Control

คลาส Control เป็นรากฐานของคอนโทรลทั้งหมด ตัวอย่างเช่น textbox หรือ button ก็คือคอนโทรลโดยทั่วไปคลาส Control จะมีฟังก์ชันและคุณสมบัติของส่วนติดต่อกับผู้ใช้ทุกตัว ทุกอย่างที่เราเห็นใน ASP.NET ก็เป็นคลาสที่สืบทอดมาจากแหล่งใดแหล่งหนึ่งทั้งสิ้น

2.3.3.1.1 คุณสมบัติของ Control

คลาส Control มีคุณสมบัติที่สำคัญดังต่อไปนี้ Control, ID, Parent, EnableViewState, Visible, Context และ ViewState คุณสมบัติของคอนโทรลเหล่านี้แทนลูกของ instace ของคอนโทรลคุณสมบัติของคลาสแม่จะบอกคลาส แม่ของคอนโทรล คุณสมบัติทั้งสองนี้ ทำให้มีลำดับชั้น(hierarchy)ของคอนโทรลในหน้าเว็บคุณสมบัติ ID ทำให้คอนโทรลถูกเข้าถึงจากการเขียนโปรแกรมได้โดยใช้ IDตามด้วยจุดและชื่อคุณสมบัติหรือเมธอดอื่นๆที่เราต้องการจะเข้าถึงของคอนโทรลนั้น เช่น MyObjectId.propertyname และยังทำให้เราสามารถเขียนตัวจัดการกับเหตุการณ์ที่เกิดขึ้นกับคอนโทรลได้ด้วย

EnableViewState เป็น flag ที่บอกว่าคอนโทรลจะยังคงเก็บview ของสถานะมันไว้หรือไม่ เช่นเดียวกับ view สถานะทั้งหมดของคอนโทรลลูกของมัน คุณสมบัติของ Context ทำให้เราสามารถดึงข้อมูลของ HTTP request ได้

ViewState เป็นอินสแตนซ์ของคลาส StateBag ซึ่งใช้ในการเก็บชื่อและค่าของข้อมูลที่สามารถสร้างให้เข้าถึงได้โดยหลาย request สำหรับหน้าเพจเดียว ชื่อและค่าเหล่านี้คืออินสแตนซ์ของคลาส StateItem

2.3.3.1.2 เมธอดในคลาส Control

เมธอดของคลาส Control นั้นมีเป็นจำนวนมาก อย่างไรก็ตามเราจะกล่าวถึงเพียงเมธอดที่มีความสำคัญ

เมธอด DataBind

เป็นการโยงคอนโทรลเข้ากับแหล่งข้อมูล เมื่อเมธอด นี้ถูกเรียก แท็กของการผูกข้อมูล <%#%> จะถูกนำมาพิจารณาใหม่เพื่อให้ข้อมูลใหม่ถูกโยงเข้ากับตำแหน่งtagที่เหมาะสม

เมธอด CreateChildControls

จะถูกเรียกก่อนที่คอนโทรลcompositional custom จะถูกจัดแสดงคอนโทรล compositional custom นั้นคล้ายกับคอนโทรล ActiveX คือมันจะจัดการสร้างคอนโทรลอื่นๆ ในการพัฒนาคอนโทรล custom เมธอดนี้สามารถถูกoverride เพื่อให้ผู้พัฒนาคอนโทรล custom สามารถสร้าง คอนโทรลลูก ก่อนทำการแสดง คอนโทรลไม่ว่าในการทำครั้งแรกหรือครั้งต่อไปก็ตาม

เมธอด Render

คล้ายกับ CreateChildControls เริ่มต้นใช้สำหรับพัฒนาคอนโทรล custom ผู้พัฒนาคอนโทรลจะ overrideเมธอดนี้ เพื่อแสดงตัวเนื้อหาในคอนโทรลผ่านทางพารามิเตอร์ HtmlTextWriter ที่เตรียมไว้

เมธอด SaveViewState และ LoadViewState

เก็บบันทึกและเรียกใช้งานสถานะของคอนโทรล คอนโทรลของเซิร์ฟเวอร์ คงสถานะของมันระหว่างการrequest ผ่านทางเมธอด นี้

2.3.3.2 คลาสเพจ

คลาสเพจนี้สืบทอดมาจากคลาส Control ซึ่งหมายความว่ามันสืบทอดมาทั้งคุณสมบัติเมธอด และ event ที่แสดงโดยคลาส Control นอกจากการสืบทอดแล้วคลาส เพจ ระบุคุณสมบัติเมธอด และ event ที่เจาะจงมากขึ้น สำหรับหน้าเว็บในเฟรมเวิร์คของ ASP.NET

ใน ASP.NET ทั้ง Application , Request , Respond , Server และ Session เป็นคุณสมบัติของคลาส เพจ นอกเหนือจากอ็อบเจกต์เหล่านี้คลาส เพจ ยังมีคุณสมบัติอื่นๆอีก เช่น Cache, ErrorPage, Postback , IsValid, Trace และ Validators

2.3.3.2.1 ตัวอย่างคุณสมบัติและเมธอดของคลาสเพจได้แก่

คุณสมบัติ Cache

ชี้ไปยังอ็อบเจกต์ cache ของขอบเขตของหน้าปัจจุบัน ในที่นี้ ทรัพยากรเช่นการติดต่อฐานข้อมูล (database connection)จะถูกเก็บเพื่อนำมาใช้ใหม่โดยไม่ต้องสร้างการติดต่อใหม่ หากว่า cache item นั้นยังไม่หมดอายุ

คุณสมบัติ ErrorPage

บอกหน้าที่จะถูกเรียกให้แสดงเมื่อมีerror เกิดขึ้น อาจใช้ @Pageแทนได้

คุณสมบัติ IsPostBack

บอกว่าการเรียกหน้าเว็บเป็นการโหลดครั้งแรกหรือไม่ หาก IsPostBack มีค่าเป็น true แสดงว่าไม่ใช่การโหลดครั้งแรก จึงไม่ควรทำการ initialize หน้าเพจซ้ำ เพื่อให้ประสิทธิภาพดีขึ้น

คุณสมบัติ Validator

รวมกลุ่มของเซิร์ฟเวอร์คอนโทรลซึ่งสามารถกำหนดระยะเวลาการทำงานของตัวเองลงในคุณสมบัติ Validatorของหน้าเว็บ เมื่อหน้าเว็บใดต้องมีการกำหนดอายุของตัวเอง ก็จะกำหนดอายุของคอนโทรลทั้งหมดในหน้านั้นแล้วกำหนดค่าที่เหมาะสมให้แก่คุณสมบัติ IsValid

คุณสมบัติ Trace

อ้างอิงจากอ็อบเจกต์ TraceContext ซึ่งใช้บอกค่าเตือนหรือข้อความบอกความผิดพลาดที่เกิดขึ้น สามารถกำหนดให้ทำการตรวจสอบหรือไม่ทำก็ได้โดยกำหนดใน web.config ซึ่งเป็นเท็กซ์ไฟล์ที่เก็บค่าขณะรันไทม์ของแอปพลิเคชันของASP.NET

2.3.4 ไวยากรณ์สำหรับ Web Form

2.3.4.1 ไคเร็กทีฟ

สำหรับ ASP แบบเก่านั้น ไคเร็กทีฟจะมีรูปแบบเป็น `%@[attribute= value]%` เพราะว่าไคเร็กทีฟใน ASP นั้นมีเพียงแบบเดียวเท่านั้น

ใน ASP.NET ได้มีการเพิ่มไคเร็กทีฟใหม่ซึ่งได้แก่ Page, Control, Import, Register, Assemble และ OutputCache และไวยากรณ์ใหม่ที่น่าสนใจคือ `<%@ directive [attribute=value] %>` โดยถ้าการประกาศกระทำโดยใช้ไวยากรณ์แบบเก่าคือไม่มีการใส่ไคเร็กทีฟ ก็จะหมายถึง ประกาศให้เป็นไคเร็กทีฟแบบปกติซึ่งก็คือ Page

2.3.4.1.1 @Page

ใน ASP.NET นอกจากจะประกอบด้วยไคเร็กทีฟแบบ ASP เก่าแล้ว ยังสนับสนุน attribute ที่สำคัญเช่น ErrorPage, Inherits, Src, และ EnableViewState อีกด้วย

ตัวอย่างการประกาศใช้งาน

```
<%@ Page Language="VB" ErrorPage = "URL" EnableViewState = "true"%>
```

2.3.4.1.2 @Control

เช่นเดียวกันกับ Page ไคเร็กทีฟ จะมีการประกาศใช้ในไฟล์ (.aspx) สำหรับ Control ไคเร็กทีฟนั้นจะใช้ใน ASP.NET Control คือประกาศในไฟล์ (.ascx)

2.3.4.1.3 @Import

เราสามารถใส่ Import ไคเร็กทีฟ ในการอ้างถึง เนมสเปซที่จะนำมาใช้ในหน้านั้นๆ ซึ่งโปรแกรมที่เราสร้างขึ้นจะสามารถเข้าถึงทุกๆ คลาสที่มีอยู่ในเนมสเปซนั้น ตัวอย่างเช่นถ้าเราต้องการใช้งาน ADO.NET เราต้องใส่ไคเร็กทีฟดังนี้คือ

```
<%@ Import Namespace="System.Data" %>
```

```
<%@ Import Namespace="System.Data.OleDb" %>
```

2.3.4.1.4 @Register

เป็นไคเร็กทีฟที่ใช้ ลงทะเบียนการใช้งานเซิร์ฟเวอร์คอนโทรลในหน้า Page นั้นๆ ตัวอย่างเช่น

```
<%@ Register Tagprefix = "Ch07"
```

```
TagName = "MyCustomControl"
```

```
Src = "MyCustomControl.ascx" %>
```

โดยชื่อของคอนโทรลคือ MyCustomControl และ คำนำหน้าเมื่อการประกาศใช้คอนโทรลคือ Ch07 ที่อยู่ของคอนโทรลคือ MyCustomControl.ascx

2.3.4.1.5 @Assembly

เป็นไคเร็กทีฟที่ใช้บ่งบอกว่า Page นั้นๆ เป็นของ assemble ไคตัวอย่างเช่น

```
<%@ Assembly Name = "Ch07" %>
```

หมายความว่า โปรแกรมในหน้า Page นี้สามารถเข้าถึงอะไรก็ได้ใน assembly Ch07

2.3.4.1.6 @OutputCache

เราสามารถใส่ Output Cache ไคเร็กทีฟในการควบคุม ระยะเวลาของ output caching ระหว่างที่หน้า Page นั้นเปิดขึ้น

2.3.4.2 ขอบเขตการประกาศคำสั่ง

เช่นเดียวกับ ASP ในการประกาศขอบเขตคำสั่งทำงาน จะหมายถึงการกำหนด ขอบเขตคำสั่งที่จะถูกประมวลผลและสั่งให้ทำงานในหน้า Page นั้นๆ ซึ่งในขอบเขตนั้นจะมี คุณลักษณะที่เรียกว่า “runat” ซึ่งเป็นการใช้กำหนดว่า ขอบเขตของคำสั่งเหล่านี้ จะประมวลผลที่ฝั่งไคลเอนต์ หรือ ฝั่งเซิร์ฟเวอร์ ซึ่งถ้าต้องการให้มีการทำงานที่ฝั่งเซิร์ฟเวอร์ จะต้องมีการกำหนดให้มี คุณลักษณะเป็น “server” และถ้าไม่มีการกำหนดจะหมายความว่าให้ทำงานในฝั่งไคลเอนต์ ตัวอย่างเช่น

```
<script runat="server" [language="codelanguage"]>
    'code
</script>
```

สำหรับการทำงานทั้งสองแบบเราสามารถกำหนดที่อยู่ของคำสั่งได้ โดยการทำเช่นนี้เป็นการแยกส่วนของคำสั่งออกจากส่วนของเอกสาร HTML การประกาศกระทำโดย

```
<script runat="server"
    [language="codelanguage"]
    [src="externalfilename"]/>
```

2.3.4.3 HTML-Control Syntax

HTML คอนโทรลมีความคล้ายกับมาตรฐาน HTML ปกติอยู่มาก จะแตกต่างกันที่มีการเพิ่มการประกาศ “id” และ “runat” แอททริบิวต์ ซึ่งโดยปกติการใช้ id กับการเขียนโปรแกรมนั้นจะเจอกับการเขียนในลักษณะ Dynamic HTML จะแตกต่างกันตรงที่ HTML คอนโทรล จะเป็นลักษณะของ คอนโทรล ที่มีทำงานในฝั่งเซิร์ฟเวอร์ ตัวอย่างเช่น

```
<input id="cmd1" runat="server"
    type="button" value="Click Me" />
```

โดยสังเกตว่าการประกาศต้องเป็นแบบ server เพราะว่าเว็บฟอร์มจะมีการใช้ POST เมธอดในการควบคุม เรื่องสแตการทำงานเมื่อทำการประมวลผลคำสั่ง ASP.NET จะทำการสร้างสคริปต์ขึ้นจากอ็อบเจกต์ HtmlControl จากตัวอย่างข้างต้น ASP.NET จะทำการสร้างคอนโทรลขึ้นจาก HtmlInputButton โดยมี id

คือ cmd1 นอกจากนี้ยังสามารถควบคุม อีเวนต์ของคอนโทรลได้อีกด้วย โดยการประกาศจะกระทำผ่าน HTML Tag

ตัวอย่างเช่น ถ้าต้องการควบคุมอีเวนต์ onclick สามารถเพิ่มในส่วนของ HTML Tag ดังนี้

```
Onserverclick = "handleServerClick"
```

2.3.4.4 Custom-Control Syntax

ในส่วนของ คอนโทรลที่กำหนดเอง ก็จะคล้ายกับ HTML เซิร์ฟเวอร์คอนโทรล โดย Custom คอนโทรลจะมีส่วนของ id และ runat แอททริบิวต์ แต่อย่างไรก็ตาม Custom คอนโทรลก็ไม่เหมือนกับ HTML มาตรฐาน ในการที่จะใช้งานเราสามารถประกาศตามรูปแบบดังนี้คือ

```
< tagprefix:tagname id="controlID" runat="server" eventname="eventHandler" />
```

ในการเรียกใช้งาน จะต้องใส่ prefix เป็น asp ดังตัวอย่างต่อไปนี้

```
<asp:TextBox id=txt1 runat=server></asp:TextBox>
```

2.3.4.5 Data-Binding Expressions

การผูกข้อมูลหรือ Data Source เข้ากับคอนโทรลใดๆ เราสามารถทำได้ตามไวยากรณ์ดังนี้คือ

```
<%# data-binding-expression %>
```

โดยการใช้ Data Bind นั้น Web Form จะไม่ทำการ Bind ให้อัตโนมัติ เราจะต้องทำการใส่เมธอด DataBind() เองภายใน โค้ดโปรแกรม

ตัวอย่างการใช้ Data Binding ดังนี้คือ

```
<asp:Label text='<%# TestData %>' runat=server/>
```

```
<html>
```

```
<head><title>Data Binding</title></head>
```

```
<body>
```

```
<script language="C#" runat= server>
```

```
/*declare the variable we want to bind to.
```

```
Public string TestData;
```

```
Void Page_Load(Object oSender, EventArgs oEvent) {
```

```
    TestData = "HelloWorld";
```

```
    Page.DataBind();
```

```
}
```

```
</script>
```

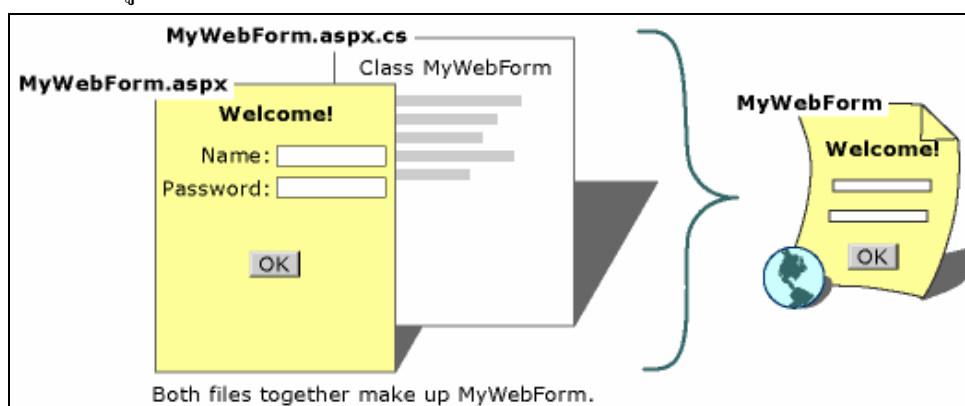
```
</body>
```

```
</html>
```

2.3.5 การพัฒนาเว็บแอปพลิเคชันด้วย ASP.NET

2.3.5.1 ส่วนประกอบคอมโพเนนต์ของ Web Form

ใน Web Forms pages นั้น การเขียน program ของ user interface แบ่งเป็นสองส่วนคือ visual Component และ logic ซึ่งก็คล้ายกับการที่ Visual Basic มีการแบ่งระหว่างส่วนของ form และส่วน code เบื้องหลังที่ form นั้นทำงานด้วยนั่นเอง ส่วนแสดงผลที่มองเห็นได้นั้นก็คือ Web Form page ซึ่งประกอบด้วยไฟล์ ที่มี HTML หรือ ASP.NET server control. Web Form page ทำงานเหมือนเป็น container สำหรับ static text และส่วนควบคุมที่เราต้องการแสดงผล ในส่วน logic ของ Web Form page ประกอบด้วย โค้ด ที่เรา สร้าง ขึ้นเพื่อสื่อสารกับ form logic การเขียนโปรแกรม จะอยู่แยกกับส่วน file ของ user interface ซึ่งไฟล์นี้จะ เปรียบเสมือน โค้ดที่อยู่เบื้องหลังการทำงาน สามารถเขียนด้วยภาษา Visual Basic หรือ Visual C#



รูปที่ 2-4 ส่วนประกอบคอมโพเนนต์ของ Web Form

ไฟล์ที่ประกอบด้วยคลาสต่างๆและ เบื้องหลังการทำงาน สำหรับ Web Form ทั้งหมดจะถูก คอมไพล์เป็น ไฟล์ dynamic-link library(.dll) page ที่ เป็น .aspx ก็จะถูก compile เช่นกัน แต่มี กระบวนการที่ต่างกัน ในครั้งแรกที่ ผู้ใช้ เรียก file หน้า .aspx ASP.NET จะสร้างไฟล์ .NET class มาเป็น ตัวแทนของ page โดยอัตโนมัติ แล้ว คอมไพล์ เป็น ไฟล์ .dll ตัวที่สอง class ที่ถูกสร้างสำหรับ .aspx page สืบทอดคุณสมบัติมาจาก class ที่เป็น โค้ดการทำงานเบื้องหลังที่ถูก compile ไปแล้วก่อนหน้านี้ เมื่อผู้ใช้ ร้องขอ URL ของ web page ไฟล์ .dll จะ run บน server และสร้างผลเป็น HTMLแบบไดนามิกสำหรับ page ที่เรียก

2.3.5.2 วงจรชีวิตของ Web Form

ใน ASP เว็บเพจจะเริ่มต้นวงจรชีวิตเมื่อไคลเอนต์เริ่มทำการร้องขอเพจที่เจาะจงมา IIS ทำการ กระจายค่าและทำการใช้งานสคริปต์บน ASP เพจ เพื่อทำให้เป็นเนื้อความ HTML ในที่สุดการทำการ แปลงก็จะสิ้นสุดอายุของเพจนั่นก็จะสิ้นสุด ถ้าคุณมีฟอร์มที่ส่งข้อมูลกลับไป ASP เพจก็จะเริ่ม

กระบวนการใหม่เหมือนกับ ASP กระทำคำร้องใหม่ โดยไม่ต้องรู้เกี่ยวกับขั้นตอนก่อนหน้าเลย การส่งผ่านข้อมูลกลับไปที่เราเริ่มต้นสำหรับการประมวลผลก็เป็นการถูกอ้างแบบโพสต์แบ็ก(postback)ด้วย

ใน ASP.NET มีหลายสิ่งที่แตกต่างกันเล็กน้อย เรายังคงเริ่มที่การร้องขอของไคลเอนท์อย่างไรก็ตามมันจะยังคงอยู่เรื่อยๆเท่าที่ไคลเอนท์ยังคงติดต่อกับเราอยู่ เพื่อให้เข้าใจขึ้นอาจจะกล่าวได้ว่าเรายังคงอยู่ไปเรื่อยแต่ในความเป็นจริงมีเพียงขั้นตอนการดู(view)ของเราระหว่างการร้องขอเท่านั้น ขั้นตอนการดูเท่านั้นที่อนุญาตให้ควบคุมบนเซิร์ฟเวอร์ที่จะปรากฏว่า ถ้ามันยังคงครองอีเวนต์ของเซิร์ฟเวอร์ เราสามารถค้นหาขั้นตอนการโพสต์แบ็กของเราผ่านคุณสมบัติ IsPostBack ของเพจอ็อบเจกต์และการกำหนดค่าเริ่มต้นใหม่ การครองอีเวนต์ระหว่างการโพสต์แบ็กมันก็คือการทำ ASP.NET แตกต่างจากการพัฒนา ASP ตามแบบธรรมดาอย่างมาก

จากตัวอย่างเราทำการขยายส่วนต้น เช่นการครองโพสต์แบ็ก เมื่อทำการโหลดอีเวนต์ที่ถูกถือครองครั้งแรก เราจะทำการย้ายข้อมูลลงใน drop-down list box หลังจากนั้นเราจะระบุเวลาของอีเวนต์ที่เพิ่มขึ้นโดยไม่มีการโหลดข้อมูลเข้ามาใหม่เท่านั้น ตัวอย่างนี้แสดงให้เห็นว่าเซิร์ฟเวอร์อีเวนต์แฮนด์เลอร์(handler)ที่ชื่อ handlerButtonClick เป็นขอบเขตของอีเวนต์ที่ชื่อ ServerClick ของปุ่ม

```
<html>

<head><title>Testing Page Event</title></head>

<body>

<script language="C#" runat=server>

    void Page_Init(Object oSender,EventArgs oEvent){

        labelInit.Text = DateTime.Now.ToString( );

    }

    void Page_Load (Object oSender, EventArgs oEvent) {

        labelInit.Text = DateTime.Now.ToString( );

        if (!IsPostBack){

            selectCtrl.Items.Add("Acrua");

            selectCtrl.Items.Add("BMW");

            selectCtrl.Items.Add("Cadillac");

            selectCtrl.Items.Add("Mercedes");

            selectCtrl.Items.Add("Porsche");

        }else{

            labelLoad.text += "(Postback)";

        }

    }

}
```



```

void handleButtonClick(Object oSender, EventArgs oEvent){
    labelOutput.Text = "You've selected: "+ selectCtrl.Value;
    labelEvent.Text = DateTime.Now.ToString ();
}
</script>
<form runat = server>
    Init Time: <asp:Label id=labelInit runat=server/><br/>
    Load Time: <asp:Label id=labelLoad runat=server/><br/>
    Event Time: <asp:Label id=labelEvent runat=server/><br/>
    Choice: <select id=selectCtrl runat=server/><br/>
    <asp:Label id=labelOutput runat=server/></select><br/>
    <input type=button value=update
        onServerClick="handleButtonClick" runat=server />
</form>
</body>
</html>

```

วงจรชีวิตของเว็บฟอร์ม ประกอบด้วยสามขั้นตอนหลัก การกำหนดรูปแบบ การถือครองอีเวนต์ และการสิ้นสุด โดยขั้นตอนเหล่านี้จะข้ามผ่านคำร้องขอที่เหมือนกันซึ่งแย้งกับนโยบาย serving-one-page-at-a-time ที่พบใน ASP

2.3.5.2.1 การกำหนดรูปแบบ

ขั้นตอนการกำหนดรูปแบบนี้ โหลดอีเวนต์ของเพจจะถูกยกขึ้นมั่นเป็นงานของคุณในการถือครองอีเวนต์นี้เพื่อติดตั้งเพจคุณ เพราะว่าโหลดอีเวนต์จะถูกยกขึ้นเมื่อการควบคุมพร้อมแล้วงานคุณตอนนี้คือการอ่านและทำการอัปเดตคุณสมบัติการควบคุมเหมือนเป็นส่วนของการติดตั้งเพจ ในตัวอย่างโค้ดข้างต้น เราจะถือครอง โหลดอีเวนต์เพื่อย้ายข้อมูลบางอย่างลง drop-down list เราอัปเดตข้อความของการควบคุม labelLoad เพื่อแสดง เวลาโหลดอีเวนต์ที่เกิดขึ้น ในแอปพลิเคชันของคุณ คุณจะต้องนำข้อมูลที่เป็นไปได้จาก ฐานข้อมูลและทำการกำหนดค่าเริ่มต้นฟอร์มฟิลด์ด้วยค่าดีฟอลต์

คุณสมบัติ IsPostBack ของเพจแสดงว่าเพจแรกได้ถูกโหลดหรือมั่นเป็นการโพสต์แบคเช่นถ้าคุณควบคุมเนื้อหารายการของข้อมูลคุณจะต้องโหลดการควบคุมเพจแรกที่ถูกโหลดโดยการตรวจสอบคุณสมบัติ IsPostBackของเพจนี้นั้นเมื่อ IsPostBack มีค่าเป็นจริงคุณจะรู้ว่ารายการควบคุมอ็อบเจกต์ได้ถูกโหลดด้วยข้อมูลเรียบร้อยแล้ว ไม่จำเป็นที่ต้องย้ายข้อมูลในรายการใหม่ตัวอย่างโค้ดข้างต้นเราจะข้ามการย้ายข้อมูลของ drop-down และแสดงเพียง สตริงว่า "(postback)"

คุณอาจจะต้องกระทำการผูก(bind)ข้อมูลและกำหนดค่าใหม่ data-binding expressions ในตอนแรก และลำดับต่อมาของเพจนี้

2.3.5.2.2 การถือครองอีเวนต์

ในขั้นตอนกลางนี้ ฟังก์ชัน server event-handlingของเพจจะถูกเรียกเป็นผลของบางอีเวนต์เป็นการถูกเรียกให้ทำงานจากฝั่งไคลเอนต์อีเวนต์เหล่านี้มาจากการควบคุมที่คุนแทนที่บนเว็บฟอร์ม

2.3.5.2.3 การจบการทำงาน

ที่ขั้นตอนนี้เพจจะสิ้นสุดการแปลงและถูกกำจัดทิ้งแล้วสิ่งที่คุณต้องรับผิดชอบคือการทำความสะดวกไฟล์แฮนด์เลอร์ ยกเลิกการติดต่อกับฐานข้อมูลและปล่อยอ็อบเจกต์ถึงแม้คุณจะเชื่อมั่นการทำงานของตัวกำจัดขยะ(garbage collection)ของ CLR ก็ตามแต่เราขอยืนยันอย่างยั้งยวดว่าควรจำทำความสะอาดด้วยตัวเองเพราะตัวกำจัดขยะจะทำงานเป็นระยะๆเท่านั้น บนการทำงานหนักๆถ้าตัวกำจัดขยะไม่เหมาะสมจะมีทรัพยากรที่ไม่ได้คืนให้ระบบทำให้หน่วยความจำหมดไปได้และทำให้ระบบคุณค้างไป

เราสามารถทำความสะอาดอย่างข้างต้นได้ด้วยการอันโหนดอีเวนต์แฮนด์เลอร์ดังที่จะแสดงต่อไปนี้เพราะว่าไม่มีแสดงในตัวอย่างเราจะแสดงเฉพาะฟังก์ชันเป็น เทมเพลต

```
Void Page_Unload(Object oSender,Eventargs oEvent){
    //cleaning up code here
}
```

2.3.6 การจัดการ State และ Scalability

2.3.6.1 ASP.NET Session-State Management

ASP.NET ได้ปรับปรุงการจัดการเรื่องสแตทใน ASP เดิม โดยปรับปรุงมาเป็นลักษณะโมเดลแบบ out-of-process โดยการจัดการสแตทจะมองลักษณะของฟอร์มของ Web Server โดยการสั่งให้มีการสลับไปยังเพจใดๆ ในฟอร์มจะไม่เกิดการหายของเซสชัน

ในการเปลี่ยนแปลงค่าเกี่ยวกับการจัดการการบริหาร session ใน ASP.NET เราสามารถกระทำผ่านไฟล์ที่ชื่อว่า web.config โดยความเป็นจริงแล้วเราสามารถทำการกำหนดค่าได้สองทางได้แก่ทางระดับ แมทชีน และระดับ แอปพลิเคชัน โดยในระดับแมทชีนสามารถกำหนดผ่านไฟล์ที่อยู่ในที่

WinNT\Microsoft. NET\Framework\<version>\CONFIG\machine.config ในขณะที่ในระดับแอปพลิเคชันสามารถกำหนดผ่าน web.config ที่อยู่ในไคลเอนต์หรือเซิร์ฟเวอร์เดียวกับแอปพลิเคชัน

โดยโค้ดที่อยู่ในไฟล์ web.config มีดังนี้คือ

```

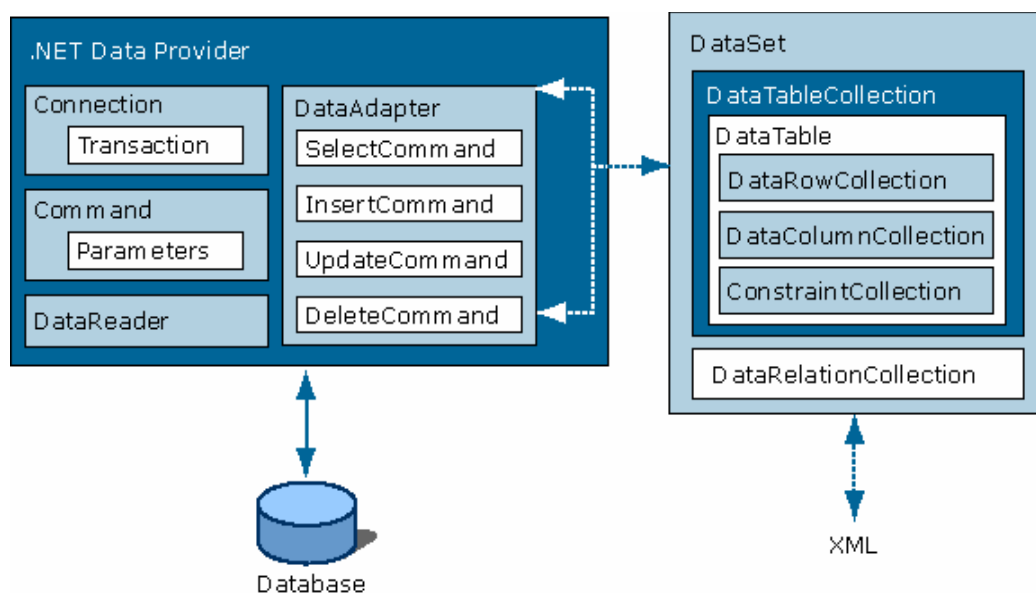
<configuration>
  <system.web>
    <sessionstate
      mode="Inproc"
      cookieless="false"
      timeout="20" />
  </system.web>
</configuration>

```

2.4 ADO.NET

2.4.1 สถาปัตยกรรม ADO.NET

แต่เดิมการประมวลผลข้อมูลนั้นเป็นแบบ Connection Based ในสถาปัตยกรรม 2-Tier ต่อมาการประมวลผลข้อมูลมีปริมาณที่มากขึ้นในสถาปัตยกรรมแบบ Multi-Tier โปรแกรมเมอร์ ต้องเปลี่ยนไปจัดการแบบ Disconnected เพื่อที่จะทำให้สามารถขยายขีดจำกัดโดยที่รองรับการใช้งานได้มากขึ้น ในอีกประเด็นหนึ่งคือ ปัจจุบันการส่งข้อมูลเป็นลักษณะผ่าน Web มากขึ้นภาษา XML มีบทบาทมากขึ้น โดยส่วนของ ADO.NET ก็จะสามารถในการรับส่งข้อมูลเป็น XML อีกด้วย ซึ่งจะอธิบายในหัวข้อต่อไป



รูปที่ 2-5 แสดงสถาปัตยกรรม ADO.NET

2.4.2 ส่วนประกอบคอมโพเนนต์ของ ADO.NET

ADO.NET ประกอบด้วย 2 คอมโพเนนต์หลักคือ DataSet และ .NET data provider ซึ่งประกอบด้วย กลุ่มของ คอมโพเนนต์คือ Connection, Command, DataReader และ DataAdapter ดังรูปที่ 1

ADO.NET DataSet เป็นคอมโพเนนต์หลักในการเชื่อมต่อข้อมูลแบบ Disconnected, DataSet ถูกออกแบบมาให้มีความเป็นอิสระจากแหล่งข้อมูลด้วยเหตุนี้ มันจึงสามารถใช้ได้ใน Data Source หลายประเภท

DataSet ประกอบด้วยกลุ่มของ DataTable ซึ่ง Table ที่ว่านี้ก็เกิดมาจากการส่ง Query ไปดึงมาจาก ฐานข้อมูล โดย DataTable ก็จะถูกประกอบไปด้วย DataRow, DataColumn รวมถึง primary key, foreign key, ข้อจำกัดต่างๆ พวก Data

Integrity และ ความสัมพันธ์ของข้อมูล (DataRelation) ใน DataTable เอง ในส่วนของการส่งข้อมูล DataSet ได้ใช้ XML เป็นตัวกลางในการส่งข้อมูลระหว่าง Tier ทำให้สามารถนำไปใช้กับการทำ Web Service ได้

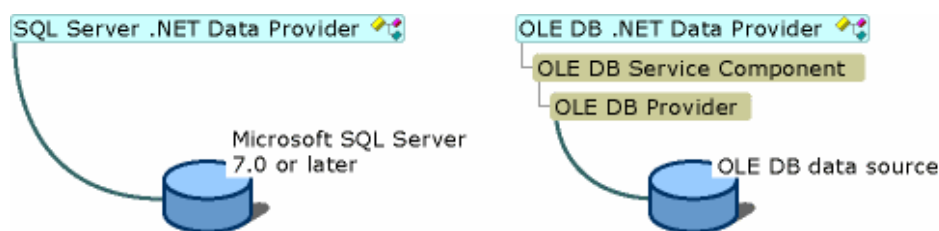
ส่วนหลักอีกส่วนหนึ่งคือ .NET Data Provider ซึ่งสร้างมาเพื่อการประมวลผลข้อมูลแบบ รวดเร็ว และแบบส่งไปข้างหน้าอย่างเดียว (forward-only) และเป็นการเข้าถึงข้อมูลแบบอ่านอย่างเดียว (read-only) โดยประกอบด้วยส่วนย่อยๆ ดังต่อไปนี้คือ

1. Connection จัดการเกี่ยวกับเรื่องการติดต่อกับ Data Source หรือ ฐานข้อมูลนั่นเอง
2. Command ทำเกี่ยวกับเรื่องการเข้าใช้งานฐานข้อมูลโดยใช้คำสั่งภาษา SQL รวมถึงการ ทำ stored procedure และ รับส่งข้อมูลพารามิเตอร์ต่างๆ
3. DataReader จัดการเรื่องประสิทธิภาพของการส่งข้อมูลจาก Data Source โดย DataReader นั้นใช้ได้ในกรณี
ที่เราดึงข้อมูลมาทีละรายการโดยไม่ย้อนกลับเท่านั้น และเป็นการดึงข้อมูลมาอย่างเดียว เขียนกลับไปไม่ได้
4. DataAdapter จะเป็นเหมือนสะพานเชื่อมต่อระหว่าง DataSet และ Data Source

โดยการทำงานคือเมื่อมีการเปิด Connection และกำหนด Query String SQL แล้ว DataAdapter ใช้ Command Object เพื่อประมวลผลคำสั่งภาษา SQL และทำการดึงข้อมูลมาที่ DataSet เพื่อใช้งานต่อไป โดย .NET Framework มี .NET data provider ให้ใช้ 2 แบบคือ

1. SQL Server .NET data provider
2. OLE DB .NET data provider

เปรียบเทียบระหว่าง SQL Server .NET Data Provider และ OLEDB .NET Data Provider



รูปที่ 2-6 เปรียบเทียบระหว่าง SQL Server .NET Data Provider
และ OLEDB .NET Data Provider

ก็คือ ถ้าใช้ SQL Server .NET Data Provider ต้องใช้กับ Microsoft SQL Server 7.0 หรือใหม่กว่าเท่านั้น โดย คลาสของ SQL Server .NET Data Provider กำหนดโดย namespace System.Data.SqlClient สำหรับเวอร์ชันที่เก่ากว่าก็ใช้ OLE DB .NET Data Provider แทน

การเลือกใช้งาน DataReader หรือ DataSet

ในการเลือกใช้ DataReader หรือ DataSet ใน แอปพลิเคชันนั้น ควรพิจารณาถึงชนิดของหน้าที่การทำงานของแอปพลิเคชัน โดยจะเลือกใช้ DataSet ก็ต่อเมื่อ

- มีการส่งข้อมูลระหว่าง-tier หรือมีต้องมีการใช้ XML Web service
- มีการติดต่อข้อมูลแบบไดนามิก โดยผูกติดกับ Window Form หรือ การรวมข้อมูลที่เกี่ยวข้องกันมาใช้งานจากแหล่งข้อมูลหลายๆที่
- ให้มีการเก็บข้อมูลลงในแคชของเครื่อง
- มีการประมวลผลข้อมูลโดยไม่ต้องเปิดการเชื่อมต่ออยู่ตลอดเวลา ซึ่งทำให้ไคลเอนต์อื่นๆ สามารถเข้ามาใช้งานฐานข้อมูลได้มากขึ้น

โดยถ้าไม่มีความจำเป็นเหล่านี้ ก็สามารถใช้ DataReader แทนได้ โดยการทำงานจะเป็นแบบ Forward-Only and read-only คือเป็นการส่งข้อมูลให้อย่างเดียว และส่งไปข้างหน้าเท่านั้น และเป็นแบบอ่านได้อย่างเดียว คือ Update ข้อมูลกลับไปไม่ได้ โดยการใช้ DataReader จะเป็นการประหยัดหน่วยความจำลงไปได้มาก

2.4.3 สรุปข้อเปรียบเทียบระหว่าง ADO และ ADO.NET

1. ในการใช้งานกับ Web ซึ่งเป็นลักษณะ ดึงข้อมูลเว็บเพจนั้นเสร็จแล้วก็ทำการตัดสายอัตโนมัติ ซึ่งส่งผลทำให้การทำงานเร็วขึ้น ใ้ว่า ADO จะไม่มี แต่ ADO.NET ทำให้มันสะดวกขึ้นโดยทำโดย

อัตโนมัติแทนที่โปรแกรมเมอร์ต้องไปควบคุมทุกขั้นตอน

หมายเหตุ (ADO.NET จะทำ snapshot ของฐานข้อมูลที่เรากำลังติดต่ออยู่เก็บไว้ในหน่วย

ความจำของเครื่อง โดยการทำ snapshot นั้นไม่ได้ทำทั้งฐานข้อมูล แต่ทำเฉพาะส่วนที่เรา query เท่านั้น โดยเมื่อดึงข้อมูลเสร็จก็จะทำการ Disconnect ทันที)

2. ส่วนการติดต่อกับฐานข้อมูลและส่วน การจัดการ client cursor (ภาษา ADO เรียกว่า Recordset) นั้นจะเป็นอิสระต่อกันโดยสิ้นเชิง ทำให้ทั้งสองส่วนพัฒนาได้โดยอิสระ
3. ใน ADO เมื่อก่อน Recordset ไม่สามารถพัฒนาได้มากนักเพราะต้องผูกติดกับ Database Server ที่ตัวเองติดต่อด้วย ดังนั้นเพื่อให้สามารถรองรับได้กับ Database ทุกประเภท ทาง Recordset จึงจำเป็นต้องถูกออกแบบมาให้ทำงานเป็นแบบกลางๆ ทำให้ความสามารถค่อยลงไป ดังนั้น ใน ADO.NET จึงแยกในส่วนการติดต่อ Database ออกมา โดยจะมีหนึ่งคลาสต่อหนึ่งต่อหนึ่งประเภท Database เช่น ถ้าเราต่อกับ Oracle ต้องใช้คลาส หนึ่ง และถ้าต่อกับ Access ต้องใช้อีกคลาสไม่เหมือนกับ ADO ที่ใช้ OLEDB , คลาส ADODB.Connection เพื่อติดต่อกับทุก Database ข้อดีข้อการแยกก็จะทำให้การเชื่อมต่อกับ DBMS แต่ละยี่ห้อเป็นไปได้โดยอิสระและสามารถดึงเอาประสิทธิภาพมาใช้ได้อย่างเต็มที่
4. ส่วนของ Client Cursor นั้นปรับปรุงใหม่ แทนที่จะเป็น Recordset ซึ่งรองรับเพียงแค่ตารางเดียวเหมือนเมื่อก่อน แต่ใน ADO.NET สามารถรองรับทั้ง database และแต่ละ table สามารถมี reference integrity ด้วย
5. การส่งผ่านข้อมูลระหว่าง ADO.NET และ DBMS นั้น แทนที่จะส่งโดยใช้ DCOM ซึ่งจะมีปัญหากับ Firewall ทำให้การสื่อสารยุ่งยาก แต่ ADO.NET นั้นจะแปลงข้อมูลอยู่ในรูปของ XML ก่อนแล้วค่อยส่งออกไป ทำให้ใช้ port ของ web มาตรฐาน ทะลุกำแพง firewall ได้

2.5 SQL 2000 Server

SQL Server 2000 เป็นโซลูชันฐานข้อมูลและการวิเคราะห์ที่สมบูรณ์แบบ ซึ่งให้ประสิทธิภาพ, ความสามารถในการขยายระบบ และมีความเชื่อถือได้ ซึ่งเป็นสภาพแวดล้อมที่สายธุรกิจขององค์กรและการทำธุรกิจบนอินเทอร์เน็ตต้องการ ด้วยความสามารถใหม่ๆ ในการสนับสนุน XML และ HTTP ช่วยให้การเข้าถึงข้อมูลและแลกเปลี่ยนข้อมูลเป็นไปโดยง่าย ขณะเดียวกันก็ให้ความสามารถในการวิเคราะห์ที่ทรงประสิทธิภาพ จึงช่วยเพิ่มคุณค่าของข้อมูลให้มากขึ้น สามารถให้บริการอย่างต่อเนื่องได้ยาวนานสูงสุด มีฟังก์ชันด้านการจัดการที่ล้ำหน้า ช่วยให้การงานประจำดำเนินไปอย่างอัตโนมัติ และมีเครื่องมือในการเขียนโปรแกรมและเซอรัวิสที่ได้รับการปรับปรุงให้ดีขึ้น ช่วยให้การพัฒนาดำเนินไปได้อย่างรวดเร็ว

นอกจากนั้นข้อดีของ SQL Server มีรายละเอียดดังต่อไปนี้

- SQL Server 2000 เป็นแกนหลักสำคัญในการจัดการและวิเคราะห์ข้อมูล สำหรับแอปพลิเคชัน และเซอรัวิสจาก Microsoft .NET ซึ่งกำลังจะเข้าสู่ผู้ใช้ในอนาคตอันใกล้
- ทำให้การรวมกันของระบบ back-end และการส่งข้อมูลข้ามไฟร์วอลล์เป็นไปโดยง่าย โดยใช้ฟังก์ชันการทำงานและการสนับสนุน XML ของ SQL Server 2000 ที่พร้อมสำหรับมาตรฐานอื่นๆ บนอินเทอร์เน็ต เช่น XPath, XSL และ XSLT ผู้พัฒนาเว็บสามารถเข้าถึงข้อมูลได้ด้วย

XML โดยไม่ต้องเขียนโปรแกรมบนฐานข้อมูลแบบสัมพันธ์อีก ในขณะที่ผู้บริหารฐานข้อมูลสามารถจัดการข้อมูลในรูปแบบ XML ได้อย่างง่ายดาย โดยใช้ Transact-SQL (T-SQL) และ stored procedure

- เชื่อมโยงไปยังฐานข้อมูล SQL Server 2000 และ OLAP cube ได้อย่างคล่องตัว โดยอาศัยเว็บและไม่ต้องเขียนโปรแกรมเพิ่มเติม มีการเชื่อมโยงฐานข้อมูลผ่านทาง HTTP ที่ได้รับการรักษาความปลอดภัย เพื่อให้แน่ใจว่าสามารถเรียกใช้ข้อมูลได้ผ่านทาง URL และมีส่วนติดต่อกับผู้ใช้ (UI) ที่เข้าใจได้ง่าย แม้ผู้ใช้จะเป็นนักพัฒนามือใหม่ก็ตาม